

포로그래머스 SQL 고득점 Kit

문제 유형

- SELECT
- SUM, MAX, MIN
- GROUP BY
- IS NULL
- JOIN
- String, Date

SELECT

평균 일일 대여 요금 구하기

```
-- [조건 1] SUV 자동차 평균 일일 대여 요금  
-- [조건 2] 평균은 숫 첫 번째 자리에서 반올림  
-- [조건 3] 컬럼명은 AVERAGE_FEE
```

```
SELECT  
    ROUND(AVG(DAILY_FEE)) as AVERAGE_FEE  
FROM  
    CAR_RENTAL_COMPANY_CAR  
WHERE  
    CAR_TYPE = 'SUV'
```

과일로 만든 아이스크림 고르기

```
-- [조건 1] 총 주문량이 3,000보다 높은 아이스크림의 맛을 조회  
-- [조건 2] 주 성분이 과일인 맛의 총주문량  
-- [조건 3] 총 주문량이 큰 순서대로 조회
```

```
SELECT  
    FH.FLAVOR  
FROM  
    FIRST_HALF FH  
LEFT JOIN  
    ICECREAM_INFO II  
ON  
    FH.FLAVOR = II.FLAVOR  
WHERE  
    (INGREDIENT_TYPE = 'fruit_based')  
    & (TOTAL_ORDER > 3000)  
ORDER BY  
    TOTAL_ORDER DESC
```

3월에 태어난 여성 회원 목록 출력하기

```

-- [조건 1] 생일이 3월
-- [조건 2] 여성회원
-- [조건 3] 추출 컬럼 : ID, 이름, 성별, 생년월일
-- [조건 4] 전화번호가 NULL인 경우 제외
-- [조건 5] 회원ID 오름차순 정렬

SELECT
    MEMBER_ID,
    MEMBER_NAME,
    GENDER,
    DATE_FORMAT(DATE_OF_BIRTH, '%Y-%m-%d') AS DATE_OF_BIRTH -- 조건 3
FROM
    MEMBER_PROFILE
WHERE
    (DATE_FORMAT(DATE_OF_BIRTH, '%m') = 03) -- 조건 1
    & (GENDER = 'W') -- 조건 2
    & (TLNO IS NOT NULL) -- 조건 4
ORDER BY
    MEMBER_ID ASC -- 조건 5

```

서울에 위치한 식당 목록 출력하기

```

-- [조건 1] 서울에 위치
-- [조건 2] 추출 컬럼 : 식당 ID, 식당 이름, 음식 종류, 즐겨찾기수, 주소, 리뷰 평균 점수
-- [조건 3] 리뷰 평균 : 소수점 세 번째 자리에서 반올림
-- [조건 4] 평균 점수, 즐겨찾기수 기준 내림 차순

-- REST_INFO : RI, REST_REVIEW : RR

-- 작성 전략
-- REST_REVIEW 테이블에서 GROUP BY를 통해 레스토랑별 평점 AGGREGATE
-- REST_INFO 테이블과 JOIN

-- [REMARK] MySQL에서 그냥 JOIN은 INNER JOIN이다

SELECT
    RI.REST_ID,
    RI.REST_NAME,
    RI.FOOD_TYPE,
    RI.FAVORITES,
    RI.ADDRESS,
    RR.SCORE -- 조건 2
FROM
    REST_INFO RI
JOIN
    (SELECT
        REST_ID,
        ROUND(AVG(REVIEW_SCORE), 2) AS SCORE
    FROM
        REST_REVIEW
    GROUP BY
        REST_ID) RR -- 조건 3
ON
    RI.REST_ID = RR.REST_ID
WHERE
    SUBSTRING(ADDRESS, 1, 2) = '서울' -- 조건 1

```

ORDER BY

RR.SCORE DESC, RI.FAVORITES DESC -- 조건 4

흉부외과 또는 일반외과 의사 목록 출력하기

-- [조건 1] 추출 컬럼 : 이름, 의사ID, 진료과, 고용일자
-- [조건 2] 진료과가 흉부외과(CS)이거나 일반외과(GS)
-- [조건 3] 고용일자 내림차순, 이름 오름차순 정렬
-- [조건 4] 날짜 포맷 예시와 동일하게 추출

SELECT

DR_NAME,
DR_ID,
MCDP_CD,
DATE_FORMAT(HIRE_YMD, '%Y-%m-%d') AS HIRE_YMD -- 조건 1, 4

FROM

DOCTOR

WHERE

(MCDP_CD = 'CS')
OR (MCDP_CD = 'GS') -- 조건 2

ORDER BY

HIRE_YMD DESC, DR_NAME ASC -- 조건 3

인기있는 아이스크림

-- [조건 1] 추출 컬럼 : 아이스크림 맛
-- [조건 2] 추출 컬럼 : 총주문량을 기준으로 내림차순, 출하 번호를 기준으로 오름차순

SELECT

FLAVOR

FROM

FIRST_HALF

ORDER BY

TOTAL_ORDER DESC, SHIPMENT_ID ASC

강원도에 위치한 생산공장 목록 출력하기

-- [조건 1] 추출 컬럼 : 공장 ID, 공장 이름, 주소
-- [조건 2] 강원도에 위치
-- [조건 3] 공장 ID를 기준으로 오름 차순

SELECT

FACTORY_ID,
FACTORY_NAME,
ADDRESS

FROM

FOOD_FACTORY

WHERE

```
ADDRESS LIKE '강원도%'
ORDER BY
FACTORY_ID ASC
```

LIKE 사용하는 경우

장점:

- 인덱스 활용 가능:
 - ADDRESS 컬럼에 적절한 **B-Tree** 인덱스가 설정되어 있다면, MySQL은 인덱스를 사용하여 '강원도%'로 시작하는 데이터를 효율적으로 검색합니다.
 - LIKE '강원도%'는 문자열의 앞부분부터 검색하므로, 인덱스 스캔이 가능합니다.
- 가독성: 조건이 간단하고 명확해 가독성이 좋습니다.

단점:

- 인덱스 부재 시 느림:
 - ADDRESS 컬럼에 인덱스가 없으면, 전체 테이블 스캔(Full Table Scan)이 발생하여 성능 저하가 있습니다.

SUBSTRING 사요 하는 경우

장점:

- 명확한 조건 설정:
 - 조건 자체는 문자열의 앞부분이 정확히 "강원도"와 일치하는지를 비교하므로, 특정 상황에서 의도를 더 명확히 드러낼 수 있습니다.

단점:

- 인덱스 활용 불가능:
 - SUBSTRING() 함수는 컬럼 값을 변환하기 때문에, 인덱스를 사용할 수 없습니다. MySQL은 변환된 값을 기반으로 조건을 평가하므로, 전체 테이블 스캔(Full Table Scan)이 발생합니다.
- 비효율적인 연산:
 - 데이터베이스는 모든 행에 대해 SUBSTRING() 연산을 수행해야 하므로, 처리 시간이 증가합니다.

12세 이하인 여자 환자 목록 출력하기

```
-- [조건 1] 추출 컬럼 : 환자이름, 환자번호, 성별코드, 나이, 전화번호
-- [조건 2] 12세 이하
-- [조건 3] 여자 환자
-- [조건 4] 전화번호 없는 경우 'NONE' 출력
-- [조건 5] 나이 기준 내림차순, 환자이름 기준 오름차순
```

```
SELECT
    PT_NAME,
    PT_NO,
    GEND_CD,
    AGE,
    IF(TLNO IS NULL, 'NONE', TLNO) AS TLNO -- 조건 1, 4
FROM
    PATIENT
```

```

WHERE
    (AGE <= 12) -- 조건 2
    AND (GEND_CD = 'W') -- 조건 3
ORDER BY
    AGE DESC,
    PT_NAME ASC -- 조건 5

```

조건에 맞는 도서 리스트 출력하기

```

-- [조건 1] 추출 컬럼 : 도서 ID, 출판일
-- [조건 2] 2021년에 출판
-- [조건 3] 카테고리 '인문'
-- [조건 4] 출판일 기준 오름차순
-- [조건 5] 날짜는 예시와 동일 포맷으로 처리

SELECT
    BOOK_ID,
    DATE_FORMAT(PUBLISHED_DATE, '%Y-%m-%d') AS PUBLISHED_DATE -- 조건 1
FROM
    BOOK
WHERE
    (CATEGORY = '인문') -- 조건 3
    AND (DATE_FORMAT(PUBLISHED_DATE, '%Y') = '2021') -- 조건 2
ORDER BY
    PUBLISHED_DATE ASC

```

조건에 부합하는 중고거래 댓글 조회하기

```

-- [조건 1] 추출 컬럼 : 게시글 제목, 게시글 ID, 댓글 ID, 댓글 작성자 ID, 댓글 내용, 댓글 작성일
-- [조건 2] '게시물'을 2022년 10월에 작성
-- [조건 3] 댓글 작성일 기준 오름차순, 게시글 제목 기준 오름차순
-- [조건 4] 날짜는 예시 포맷과 일치

-- USED_GOODS_BOARD : board
-- USED_GOODS_REPLY : reply

SELECT
    board.TITLE,
    board.BOARD_ID,
    reply.REPLY_ID,
    reply.WRITER_ID,
    reply.CONTENTS,
    DATE_FORMAT(reply.CREATED_DATE, '%Y-%m-%d') AS CREATED_DATE -- 조건 1,4
FROM
    USED_GOODS_BOARD board
JOIN
    USED_GOODS_REPLY reply
ON
    board.BOARD_ID = reply.BOARD_ID
WHERE
    board.CREATED_DATE LIKE '2022-10%' -- 조건 2
ORDER BY

```

```
reply.CREATED_DATE ASC,  
board.TITLE ASC -- 조건 3
```

재구매가 일어난 상품과 회원 리스트 구하기

```
-- [조건 1] 동일한 회원이 동일한 상품을 재구매한 데이터를 구할 것  
-- [조건 2] 출력 컬럼 : 재구매한 회원 ID와 재구매한 상품 ID  
-- [조건 3] 회원 ID를 기준으로 오름차순, 상품 ID를 기준으로 내림차순
```

```
SELECT  
    USER_ID,  
    PRODUCT_ID -- 조건 2  
FROM  
    ONLINE_SALE  
GROUP BY  
    USER_ID, PRODUCT_ID  
HAVING  
    COUNT(*) > 1 -- 조건 1  
ORDER BY  
    USER_ID ASC, PRODUCT_ID DESC -- 조건 3
```

핵심 역할

7. 집계된 결과에 조건을 적용:

- `GROUP BY`로 그룹화된 데이터에 대해 집계 함수(`SUM`, `AVG`, `COUNT`, `MAX`, `MIN` 등)를 기반으로 필터링할 때 사용

8. `WHERE` 절과의 차이:

- `WHERE`는 그룹화 이전의 개별 행에 대한 조건을 필터링하며, 집계 함수는 사용할 수 없음
- `HAVING`은 그룹화 이후의 결과에 대해 조건을 필터링

WHERE vs HAVING 요약

특징	WHERE	HAVING
적용 시점	그룹화 이전	그룹화 이후
적용 대상	개별 행	집계된 결과
집계 함수 사용 가능 여부	불가능	가능
주요 사용 목적	개별 행 필터링	그룹화된 데이터 필터링

성능 최적화 팁

- `WHERE` 절에서 가능한 한 많은 데이터를 필터링한 후, `HAVING`을 사용하는 것이 성능 면에서 유리합니다.
 - `WHERE`로 먼저 필터링 → `GROUP BY`로 그룹화 → `HAVING`으로 최종 필터링.

모든 레코드 조회하기

```
-- [조건 1] 모든 정보 조회
```

```
-- [조건 2] ANIMAL_ID 오름차순
```

```
SELECT
    *
FROM
    ANIMAL_INS
ORDER BY
    ANIMAL_ID ASC
```

역순 정렬하기

```
-- [조건 1] 추출 컬럼 : 동물의 이름, 보호 시작일
-- [조건 2] ANIMAL_ID 역순 정렬
```

```
SELECT
    NAME,
    DATETIME
FROM
    ANIMAL_INS
ORDER BY
    ANIMAL_ID DESC
```

오프라인/온라인 판매 데이터 통합하기

```
-- [조건 1] 기간 : 2022년 3월
-- [조건 2] 오프라인/온라인 상품 판매 데이터의 추출 컬럼 : 판매 날짜, 상품ID, 유저ID, 판매량을 출력
-- [조건 3] OFFLINE_SALE 테이블의 판매 데이터의 USER_ID 값은 NULL 로 표시
-- [조건 4] 판매일을 기준으로 오름차순, 상품 ID를 기준으로 오름차순, 유저 ID를 기준으로 오름차순
-- [조건 5] 일자 포맷에 주의
```

```
(SELECT
    DATE_FORMAT(SALES_DATE, '%Y-%m-%d') AS SALES_DATE,
    PRODUCT_ID,
    USER_ID,
    SALES_AMOUNT
FROM
    ONLINE_SALE
WHERE
    SALES_DATE BETWEEN '2022-03-01' AND '2022-03-31')
UNION ALL
(SELECT
    DATE_FORMAT(SALES_DATE, '%Y-%m-%d') AS SALES_DATE,
    PRODUCT_ID,
    NULL AS USER_ID,
    SALES_AMOUNT
FROM
    OFFLINE_SALE
WHERE
    SALES_DATE BETWEEN '2022-03-01' AND '2022-03-31')
```

```
ORDER BY
  SALES_DATE ASC,
  PRODUCT_ID ASC,
  USER_ID ASC
```

힌트 1: 테이블을 Stack하는 방법

여기서는 `UNION ALL` 을 사용해 두 테이블의 데이터를 합칠 수 있습니다. 이유는 중복 제거가 필요하지 않으며, 모든 데이터를 가져와야 하기 때문입니다.

힌트 2: 두 테이블의 공통 컬럼을 맞추기

두 테이블의 구조가 약간 다르므로, `OFFLINE_SALE` 테이블에 없는 `USER_ID` 컬럼은 `NULL` 로 채워줘야 합니다. 예를 들어:

- `ONLINE_SALE` 테이블에서 가져올 컬럼: `SALES_DATE` , `PRODUCT_ID` , `USER_ID` , `SALES_AMOUNT`
- `OFFLINE_SALE` 테이블에서 가져올 컬럼: `SALES_DATE` , `PRODUCT_ID` , `NULL AS USER_ID` , `SALES_AMOUNT`

힌트 3: 2022년 3월 데이터 필터링

각 테이블에서 `SALES_DATE` 가 `2022-03-01` 부터 `2022-03-31` 사이인 데이터를 선택해야 합니다. 이는 `WHERE` 절에서 처리합니다.

힌트 4: 정렬 조건

결과를 정렬할 때는 다음 순서로 처리해야 합니다:

9. `SALES_DATE` 오름차순
10. `PRODUCT_ID` 오름차순
11. `USER_ID` 오름차순 (`NULL` 은 가장 마지막에 정렬됨)

아픈 동물 찾기

```
-- [조건 1] 추출 컬럼 : 아이디, 이름
-- [조건 2] 아픈 동물만 추출
-- [조건 3] 아이디 순으로 정렬

SELECT
  ANIMAL_ID,
  NAME -- 조건 1
FROM
  ANIMAL_INS
WHERE
  INTAKE_CONDITION = 'Sick' -- 조건 2
ORDER BY
  ANIMAL_ID -- 조건 3
```

어린 동물 찾기

```
-- [조건 1] `INTAKE_CONDITION`이 Aged가 아님
-- [조건 2] 추출 컬럼 : 아이디, 이름
-- [조건 3] 아이디 순으로 정렬
```



```
SELECT
    ANIMAL_ID,
    NAME  -- 조건 2
FROM
    ANIMAL_INS
WHERE
    INTAKE_CONDITION != 'Aged'  -- 조건 1
ORDER BY
    ANIMAL_ID  -- 조건 3
```

동물의 아이디와 이름

```
-- [조건 1] 모든 동물의 아이디와 이름
-- [조건 2] ANIMAL_ID순으로 조회
```

```
SELECT
    ANIMAL_ID,
    NAME
FROM
    ANIMAL_INS
ORDER BY
    ANIMAL_ID
```

여러 기준으로 정렬하기

```
-- [조건 1] 모든 동물의 아이디와 이름, 보호 시작일 추출
-- [조건 2] 이름 오름차순, 보호시작일 내림차순으로 조회
```

```
SELECT
    ANIMAL_ID,
    NAME,
    DATETIME
FROM
    ANIMAL_INS
ORDER BY
    NAME ASC, DATETIME DESC
```

상위 n개 레코드

```
-- [조건 1] 가장 먼저 들어온 동물 이름을 조회
```

```
SELECT
    NAME
FROM
    ANIMAL_INS
```

```
ORDER BY
    DATETIME
LIMIT
    1
```

조건에 맞는 회원수 구하기

```
-- [조건 1] 2021년에 가입한 회원
-- [조건 2] 나이가 20세 이상 29세 이하 회원이 몇 명인지

SELECT
    COUNT(*)
FROM
    USER_INFO
WHERE
    (AGE BETWEEN 20 AND 29)
    & (JOINED BETWEEN '2021-01-01' AND '2021-12-31')
```

업그레이드 된 아이템 구하기

```
-- [조건 1] 아이템의 희귀도가 'RARE'인 아이템들의 모든 다음 업그레이드 아이템의 아이템 ID(ITEM_ID), 아이템 명(
-- [조건 2] 아이템 ID를 기준으로 내림차순 정렬주세요.

-- 1. RARE인 애들 필터링
-- 2. 필터링된 데이터의 ITEM_ID를 PARENT_ITEM_ID랑 JOIN
-- 3. PARENT_ITEM_ID를 기준으로

SELECT DISTINCT
    NEW_T.ITEM_ID, T3.ITEM_NAME, T3.RARITY
FROM
    (SELECT
        IT.ITEM_ID AS ITEM_ID
      FROM
        ITEM_INFO II
      JOIN
        ITEM_TREE IT
      ON
        II.ITEM_ID = IT.PARENT_ITEM_ID -- 2번
     WHERE
        RARITY = 'RARE' -- 1번
    ) as NEW_T
JOIN
    ITEM_INFO T3
ON
    NEW_T.ITEM_ID = T3.ITEM_ID
ORDER BY
```

핵심 문법

- SELECT DISTINCT
- FROM 절에 서브쿼리 작성하기

Python 개발자 찾기

```
-- [조건 1] 파이썬 스킬을 가진 개발자
-- [조건 2] 추출 컬럼 : ID, 이메일, 이름, 성
-- [조건 3] ID 기준 오름차순 정렬

SELECT
    ID,
    EMAIL,
    FIRST_NAME,
    LAST_NAME
FROM
    DEVELOPER_INFOS
WHERE
    (SKILL_1 = 'Python')
    or (SKILL_2 = 'Python')
    or (SKILL_3 = 'Python')
    # 혹은 'PYTHON' IN (SKILL_1, SKILL_2, SKILL_3)
ORDER BY
    ID ASC
```

핵심 문법

- 복수 조건을 IN 사용해서 1줄로 표현하기

조건에 맞는 개발자 찾기

```
-- [조건 1] Python이나 C# 스킬을 가진 개발자 정보 조회
-- [조건 2] 개발자 ID, 이메일, 이름, 성 조회
-- [조건 3] 개발자 ID 기준 오름차순 정렬

-- [단계 1] WITH 절로 파이썬과 C#의 스킬코드를 가져온다
WITH TARGET_CODE AS (
    SELECT
        CODE
    FROM
        SKILLCODES
    WHERE
        NAME IN ('Python', 'C#')
)
```

--[단계 2] WHERE절에서 EXIST와 SELECT 1을 같이 사용해서 행을 필터

```
SELECT
    D.ID,
    D.EMAIL,
    D.FIRST_NAME,
    D.LAST_NAME
FROM
    DEVELOPERS D
WHERE
    EXISTS(
        SELECT
            1
        FROM
            TARGET_CODE T
        WHERE
            (D.SKILL_CODE & T.CODE) > 0 --[단계 3] 이때 비트 연산자인 & 사용해 포함여부 필터
    )
ORDER BY
    D.ID
```

(HINT) 이진수로 나눴을 때, 나머지가 0이면 해당 스킬을 보유하고 있다고 할 수 있다

설명

12. WITH TargetCodes :

- TargetCodes 는 SKILLCODES 테이블에서 NAME 이 'Python' 또는 'C#'에 해당하는 CODE 값을 추출합니다.
- 이 부분은 256 과 1024 값을 직접 입력하지 않아도 쿼리가 동적으로 동작하도록 만듭니다.

13. EXISTS 절:

- DEVELOPERS 테이블의 각 SKILL_CODE 에 대해, TargetCodes 에 포함된 CODE 와의 비트 AND 연산을 수행합니다.
- (D.SKILL_CODE & T.CODE) > 0 조건이 참이면, 해당 개발자는 Python 또는 C# 스킬을 보유한 것으로 간주됩니다.

14. 정렬:

- 결과를 ID 기준으로 오름차순 정렬합니다.

상세 설명

SELECT 1이란 무엇인가?

SQL에서 SELECT 1 은 쿼리에서 특정 행이 존재하는지 확인할 때 자주 사용됩니다. 여기서 1 은 단순히 상수 값으로, 데이터를 실제로 반환하려는 목적이 아니라 "행이 존재하는지 여부"만 확인하려는 목적으로 사용됩니다.

주요 특징:

15. 효율성:

- 데이터베이스는 실제 데이터를 반환하지 않고, 단지 "행이 존재한다"는 사실만 확인합니다.
- SELECT * 와 달리 모든 컬럼 데이터를 읽지 않으므로, 성능 면에서 약간 더 효율적입니다.

16. 결과 반환 없음:

- EXISTS 와 함께 사용되면 SELECT 1 은 결과를 출력하지 않고 단순히 존재 여부만 판단합니다.

17. 기능적 동일성:

- `SELECT 1`은 `EXISTS` 문맥에서 `SELECT *`와 동일하게 동작하지만, `SELECT 1`을 사용하면 의도가 더 명확해지고 효율적인 경우가 많습니다.

EXISTS란 무엇인가?

`EXISTS`는 SQL에서 서브쿼리가 하나 이상의 행을 반환하는지 확인하기 위한 논리 연산자입니다. 만약 서브쿼리가 하나 이상의 행을 반환하면 `EXISTS`는 `TRUE`를 반환하고, 반환된 행이 없으면 `FALSE`를 반환합니다.

기본 문법:

```
SELECT
    columns
FROM
    table
WHERE
    EXISTS (
        SELECT 1
        FROM other_table
        WHERE condition );
```

작동 원리:

18. 평가 방법:

- 메인 쿼리의 각 행에 대해, 서브쿼리가 실행됩니다.
- 서브쿼리가 하나 이상의 행을 반환하면 `EXISTS`는 `TRUE`로 평가됩니다.

19. 단축 실행 (Short-Circuit):

- 서브쿼리는 첫 번째로 조건에 맞는 행을 찾는 즉시 실행을 멈춥니다.
- 모든 행을 검색하지 않으므로 성능이 최적화됩니다.

20. *결과 반환 없음

- 서브쿼리에서 선택된 값(예: `SELECT 1` 또는 `SELECT *`)은 메인 쿼리에 반환되지 않습니다. 오직 행의 존재 여부만 평가됩니다.

비트 연산이 필요한 이유

개발자의 `SKILL_CODE`가 어떤 스킬(`T.CODE`)을 포함하는지 확인하려면:

21. 비트 AND 연산 (&):

- 두 숫자의 2진수 표현에서 해당 비트가 모두 `1`인 경우에만 결과 값이 `1`이 됩니다.
- 예:

- `SKILL_CODE = 400 (b'110010000')`

- `CODE = 256 (b'100000000')`

- `400 & 256 = 256` (스킬 포함됨)

- 반대로, `400 & 1024 = 0` (스킬 미포함).

22. 조건 `(D.SKILL_CODE & T.CODE) > 0`:

- `D.SKILL_CODE`에 `T.CODE`가 포함된 경우, AND 연산 결과는 `0`보다 큰 값이 나옵니다.
 - 포함되지 않은 경우 결과는 `0`이 됩니다.
-

잔챙이 잡은 수 구하기

```
-- [조건 1] 잡은 물고기 중 길이가 10cm 이하인 물고기의 수를 출력
-- [조건 2] 컬럼명은 FISH_COUNT
```

```
SELECT
    COUNT(ID) AS FISH_COUNT
FROM
    FISH_INFO
WHERE
    LENGTH IS NULL
```

가장 큰 물고기 10마리 구하기

```
-- [조건 1] `FISH_INFO` 테이블에서 가장 큰 물고기 10마리
-- [조건 2] 출력 컬럼 : ID, 길이
-- [조건 3] 길이내림차순, ID 오름차순 정렬해주세요.
-- [조건 4] 가장 큰 물고기 10마리 중 길이가 10cm 이하인 경우는 없음
-- [조건 5] ID 컬럼명은 `ID`, 길이 컬럼명은 `LENGTH`로 해주세요.
```

```
SELECT
    ID,
    LENGTH
FROM
    FISH_INFO
ORDER BY
    LENGTH DESC, ID ASC
LIMIT
    10
```

특정 물고기를 잡은 총 수 구하기

```
-- [조건 1] `FISH_INFO` 테이블에서 잡은 `BASS`와 `SNAPPER`의 수를 출력
-- [조건 2] 컬럼명은 FISH_COUNT
```

```
-- [방법 1] JOIN으로 풀기 : FISH_TYPE을 기준으로 조인 시킨 후 카운트
-- [방법 2] EXIST (SELECT 1 FROM WHERE) 로 풀기
```

```
-- [방법 1] JOIN으로 풀기 : FISH_TYPE을 기준으로 조인 시킨 후 카운트
SELECT
```

```

COUNT(ID) AS FISH_COUNT
FROM
  FISH_INFO FI
JOIN
  FISH_NAME_INFO FN
ON
  FI.FISH_TYPE = FN.FISH_TYPE
WHERE
  FN.FISH_NAME IN ('BASS', 'SNAPPER')

-- [방법 2] EXIST(SELECT 1 FROM WHERE) 로 풀기

WITH TARGET_TABLE AS (
  SELECT
    FISH_TYPE
  FROM
    FISH_NAME_INFO
  WHERE
    FISH_NAME IN ('BASS', 'SNAPPER')
)

SELECT
  COUNT(*) AS FISH_COUNT
FROM
  FISH_INFO FI
WHERE
  EXISTS(
    SELECT
      1
    FROM
      TARGET_TABLE T
    WHERE
      FI.FISH_TYPE IN (SELECT FISH_TYPE FROM TARGET_TABLE))

```

대장균들의 자식의 수 구하기

```

-- [조건 1] 추출 컬럼 : 대장균 개체의 ID(`ID`)와 자식의 수(`CHILD_COUNT`)
-- [조건 2] 자식이 없다면 자식의 수는 0
-- [조건 3] ID에 대해 오름차순 정렬

```

```

WITH TARGET_TABLE AS (
  SELECT
    PARENT_ID AS ID,
    COUNT(ID) AS CHILD_COUNT
  FROM
    ECOLI_DATA ED

```

```

GROUP BY
    PARENT_ID
)

SELECT
    E.ID,
    IF(T.CHILD_COUNT IS NULL, 0, T.CHILD_COUNT) AS CHILD_COUNT
FROM
    ECOLI_DATA E
LEFT OUTER JOIN
    TARGET_TABLE T
ON
    E.ID = T.ID
ORDER BY
    E.ID

```

- **LEFT OUTER JOIN**에 주의하자!

대장균의 크기에 따라 분류하기 1

```

-- [조건 1] 대장균 개체의 크기가 100 이하라면 'LOW', 100 초과 1000 이하라면 'MEDIUM', 1000 초과라면 'I
-- [조건 2] 출력 컬럼 : ID(`ID`) 와 분류(`SIZE`)
-- [조건 3] ID 에 대해 오름차순

```

```

SELECT
    ID,
    IF(SIZE_OF_COLONY <= 100, 'LOW',
        IF((SIZE_OF_COLONY > 100) & (SIZE_OF_COLONY <= 1000), 'MEDIUM',
            IF(SIZE_OF_COLONY > 1000, 'HIGH', NULL))) AS SIZE
FROM
    ECOLI_DATA
ORDER BY
    ID

# 혹은
SELECT
    ID,
    CASE
        WHEN SIZE_OF_COLONY <= 100 THEN 'LOW'
        WHEN SIZE_OF_COLONY > 100 AND SIZE_OF_COLONY <= 1000 THEN 'MEDIUM'
        WHEN SIZE_OF_COLONY > 1000 THEN 'HIGH'
        ELSE NULL
    END AS SIZE
FROM
    ECOLI_DATA
ORDER BY
    ID;

```


- CASE - WHEN - THEN - - END

특정 형질을 가지는 대장균 찾기

-- [조건 1] 2번 형질이 보유하지 않으면서 1번이나 3번 형질을 보유하고 있는 대장균 개체의 수(`COUNT`)를 출력
 -- [조건 2] 1번과 3번 형질을 모두 보유하고 있는 경우도 1번이나 3번 형질을 보유하고 있는 경우에 포함

```
SELECT
    COUNT(*) AS COUNT
FROM
    ECOLI_DATA
WHERE
    (GENOTYPE & 2) = 0 -- 2번 형질이 없는 경우
    AND ((GENOTYPE & 1) != 0 OR (GENOTYPE & 4) != 0); -- 1번 형질이나 3번 형질이 있는 경우
```

- 비트 연산 후, =0이면 해당 형질 미포함
- 비트 연산 후, !=0이면 해당 형질 포함

부모의 형질을 모두 가지는 대장균 찾기

-- [조건 1] 부모의 형질을 모두 보유한 대장균의 ID(ID), 대장균의 형질(GENOTYPE), 부모 대장균의 형질(PARENT_G
 -- [조건 2] ID에 대해 오름차순

```
SELECT
    T1.ID, T1.GENOTYPE, T2.GENOTYPE AS PARENT_GENOTYPE
FROM
    ECOLI_DATA T1
JOIN
    ECOLI_DATA T2
ON
    T1.PARENT_ID = T2.ID
WHERE
    (T2.GENOTYPE & T1.GENOTYPE) = T2.GENOTYPE #이게 핵심임
ORDER BY
    T1.ID
```

- 비트 연산 후, 연산 결과가 PARENT와 같은지 확인하면 된다.
- 즉 연산 후 결과가 PARENT가 가지고 있는 형질은 모두 가지고 있어야한다.

대장균의 크기에 따라 분류하기 2

-- [조건 1] 대장균 개체의 크기를 내림차순으로 정렬했을 때 상위 0% ~ 25% 를 'CRITICAL', 26% ~ 50% 를 'HIG
 -- [조건 2] ID(`ID`) 분류된 이름(`COLONY\_NAME`)을 출력
 -- [조건 3] ID 에 대해 오름차순
 -- [조건 4] 총 데이터의 수는 4의 배수

-- [조건 5] 같은 사이즈의 대장균 개체가 서로 다른 이름으로 분류되는 경우는 없음

```
SELECT
    ID,
    CASE
        WHEN NTILE(4) OVER (ORDER BY SIZE_OF_COLONY DESC) = 1 THEN 'CRITICAL'
        WHEN NTILE(4) OVER (ORDER BY SIZE_OF_COLONY DESC) = 2 THEN 'HIGH'
        WHEN NTILE(4) OVER (ORDER BY SIZE_OF_COLONY DESC) = 3 THEN 'MEDIUM'
        WHEN NTILE(4) OVER (ORDER BY SIZE_OF_COLONY DESC) = 4 THEN 'LOW'
        ELSE NULL
    END AS COLONY_NAME
FROM
    ECOLI_DATA
ORDER BY
    ID ASC
```

- NTILE 함수
 - NTILE(n) OVER (ORDER BY col)

특정 세대의 대장균 찾기

-- [조건 1] 3세대의 대장균의 ID(ID) 를 출력

```
WITH
    GEN2 AS (
        SELECT
            ID
        FROM
            ECOLI_DATA
        WHERE
            PARENT_ID IS NULL),

    GEN3 AS(
        SELECT
            ID
        FROM
            ECOLI_DATA
        WHERE
            PARENT_ID IN (SELECT ID FROM GEN2)
    )

SELECT
    ID
FROM
    ECOLI_DATA
WHERE
    PARENT_ID IN (SELECT ID FROM GEN3)
```

```

-- 참고

WITH RECURSIVE Generations AS (
  -- 1세대: 부모 ID가 NULL인 대장균
  SELECT
    ID,
    PARENT_ID,
    1 AS generation
  FROM
    ECOI_DATA
  WHERE
    PARENT_ID IS NULL

  UNION ALL

  -- N세대: 이전 세대의 ID를 PARENT_ID로 가지는 대장균
  SELECT
    E.ID,
    E.PARENT_ID,
    G.generation + 1 AS generation
  FROM
    ECOI_DATA E
  JOIN
    Generations G
  ON
    E.PARENT_ID = G.ID
)
-- 3세대 필터링
SELECT
  ID
FROM
  Generations
WHERE
  generation = 3
ORDER BY
  ID;

```

쿼리 설명

23. CTE 정의 (WITH RECURSIVE Generations):

- 기본 단계:
 - 부모 ID (PARENT_ID)가 NULL인 대장균을 1세대로 설정합니다.
- 재귀 단계:
 - 이전 세대의 ID를 PARENT_ID 로 가지는 대장균을 찾아 다음 세대로 설정합니다.

24. 세대 계산:

- 각 대장균에 대해 세대를 계산하며, 재귀적으로 관계를 탐색합니다.
- generation + 1 로 자식 개체의 세대를 계속 증가시킵니다.

25. 3세대 필터링:

- 최종적으로 generation = 3 인 대장균만 선택합니다.

26. 결과 정렬:

- 결과를 ID 기준으로 오름차순 정렬합니다.

멸종위기의 대장균 찾기

-- [조건 1] 각 세대별 자식이 없는 개체의 수(COUNT)와 세대(GENERATION)를 출력
-- [조건 2] 세대에 대해 오름차순 정렬
-- [조건 3] 단, 모든 세대에는 자식이 없는 개체가 적어도 1개체는 존재

```
WITH
  RECURSIVE Generations AS(
    SELECT
      ID,
      PARENT_ID,
      1 AS generation
    FROM
      ECOLI_DATA
    WHERE
      PARENT_ID IS NULL

    UNION ALL

    SELECT
      E.ID,
      E.PARENT_ID,
      G.generation + 1 AS generation
    FROM
      ECOLI_DATA E
    JOIN
      Generations G
    ON
      E.PARENT_ID = G.ID)
```

```
SELECT
  COUNT(ID) AS COUNT,
  generation as GENERATION
FROM
  Generations G
WHERE
  ID NOT IN (
    SELECT PARENT_ID
    FROM Generations
    WHERE PARENT_ID IS NOT NULL)
GROUP BY
  GENERATION
```

-- [방법 2] JOIN을 사용한 후, NULL만 필터링 하는 방법

```
WITH RECURSIVE Generations AS (
  -- 1세대: 부모 ID가 NULL인 대장균
  SELECT
```

```

        ID,
        PARENT_ID,
        1 AS generation
FROM
    ECOLI_DATA
WHERE
    PARENT_ID IS NULL

UNION ALL

-- N세대: 이전 세대의 ID를 부모 ID로 가지는 대장균
SELECT
    E.ID,
    E.PARENT_ID,
    G.generation + 1 AS generation
FROM
    ECOLI_DATA E
JOIN
    Generations G
ON
    E.PARENT_ID = G.ID
),
NoChildren AS (
    -- 자식이 없는 대장균 필터링
    SELECT
        G.ID,
        G.generation
    FROM
        Generations G
    LEFT JOIN
        ECOLI_DATA E
    ON
        G.ID = E.PARENT_ID
    WHERE
        E.ID IS NULL
)
-- 세대별 자식이 없는 대장균 수 계산
SELECT
    COUNT(*) AS COUNT,
    generation AS GENERATION
FROM
    NoChildren
GROUP BY
    generation
ORDER BY
    generation;

```

SUM, MAX, MIN

가격이 제일 비싼 식품의 정보 출력하기

```

-- [조건 1] 가격이 제일 비싼 식품

```

-- [조건 2] 추출 컬럼 : 식품 ID, 식품 이름, 식품 코드, 식품분류, 식품 가격

```
SELECT
    PRODUCT_ID,
    PRODUCT_NAME,
    PRODUCT_CD,
    CATEGORY,
    PRICE
FROM
    FOOD_PRODUCT
ORDER BY
    PRICE DESC
LIMIT
    1
```

가장 비싼 상품 구하기

-- [조건 1] 가장 높은 판매가를 출력
-- [조건 2] 컬럼명 : MAX_PRICE

-- [방법 1]

```
# SELECT
#     MAX(PRICE) MAX_PRICE
# FROM
#     PRODUCT
```

-- [방법 2]

```
SELECT
    PRICE AS MAX_PRICE
FROM
    PRODUCT
ORDER BY
    PRICE DESC
LIMIT
    1
```

최댓값 구하기

-- [조건 1] 가장 최근에 들어온 동물은 언제 들어왔는지 조회

-- [방법 1]

```
SELECT
    DATETIME AS 시간
FROM
    ANIMAL_INS
ORDER BY
    DATETIME DESC
LIMIT
```

1

```
-- [방법 2]
SELECT
    MAX(DATETIME) AS 시간
FROM
    ANIMAL_INS
```

최솟값 구하기

```
-- [조건 1] 가장 먼저 들어온 동물
SELECT
    MIN(DATETIME) AS 시간
FROM
    ANIMAL_INS
```

동물 수 구하기

```
-- [조건 1] 동물 보호소에 동물이 몇 마리 들어왔는지 조회

SELECT
    COUNT(ANIMAL_ID)
FROM
    ANIMAL_INS
```

중복 제거하기

```
-- [조건 1] 동물 보호소에 들어온 동물의 이름은 몇 개인지 조회
-- [조건 2] 이름이 NULL인 경우는 집계하지 않으며, 중복되는 이름은 하나로 칩니다.

SELECT
    COUNT(DISTINCT NAME)
FROM
    ANIMAL_INS
```

조건에 맞는 아이템들의 가격의 총합 구하기

```
-- [조건 1] ITEM_INFO 테이블에서 희귀도가 'LEGEND'인 아이템들의 가격의 총합
```

-- [조건 2] 컬럼명은 'TOTAL_PRICE'로 지정

```
SELECT
    SUM(PRICE) AS TOTAL_PRICE
FROM
    ITEM_INFO
WHERE
    RARITY = 'LEGEND'
```

물고기 종류 별 대어 찾기

-- [조건 1] 물고기 종류 별로 가장 큰 물고기의 ID, 물고기 이름, 길이를 출력
-- [조건 2] 물고기의 ID 컬럼명은 ID, 이름 컬럼명은 FISH_NAME, 길이 컬럼명은 LENGTH
-- [조건 3] ID에 대해 오름차순 정렬
-- [조건 4] 단, 물고기 종류별 가장 큰 물고기는 1마리만 있으며, 10cm 이하의 물고기가 가장 큰 경우는 없습니다.

```
WITH
    MAX_LEN AS (
        SELECT
            FISH_TYPE, MAX(LENGTH) AS LENGTH
        FROM
            FISH_INFO
        GROUP BY
            FISH_TYPE
    ),
    T_NAME AS (
        SELECT
            FI.ID, FI.FISH_TYPE, FI.LENGTH, FI.TIME, FNI.FISH_NAME
        FROM
            FISH_INFO FI
        JOIN
            FISH_NAME_INFO FNI
        ON
            FI.FISH_TYPE = FNI.FISH_TYPE
    )
```

```
SELECT
    TN.ID,
    TN.FISH_NAME,
    TN.LENGTH
FROM
    MAX_LEN ML
INNER JOIN
    T_NAME TN
ON
    (ML.FISH_TYPE = TN.FISH_TYPE)
    & (ML.LENGTH = TN.LENGTH)
ORDER BY
```


ID ASC

잡은 물고기 중 가장 큰 물고기의 길이 구하기

-- [조건 1] 가장 큰 물고기의 길이를 'cm' 를 붙여 출력하는 SQL 문을 작성
-- [조건 2] 이 때 컬럼명은 'MAX_LENGTH' 로 지정

```
SELECT
    CONCAT(MAX(LENGTH), 'cm') AS MAX_LENGTH
FROM
    FISH_INFO
```

연도별 대장균 크기의 편차 구하기

-- [조건 1] 분화된 연도(YEAR), 분화된 연도별 대장균 크기의 편차(YEAR_DEV), 대장균 개체의 ID(ID) 를 출력
-- [조건 2] 분화된 연도별 대장균 크기의 편차는 분화된 연도별 가장 큰 대장균의 크기 - 각 대장균의 크기로 구함
-- [조건 3] 결과는 연도에 대해 오름차순으로 정렬하고 같은 연도에 대해서는 대장균 크기의 편차에 대해 오름차순

```
WITH MAX_INFO AS (
    SELECT
        MAX(SIZE_OF_COLONY) AS YEAR_MAX,
        YEAR(DIFFERENTIATION_DATE) AS YEAR
    FROM
        ECOLI_DATA
    GROUP BY
        YEAR(DIFFERENTIATION_DATE)
)

SELECT
    YEAR(ED.DIFFERENTIATION_DATE) AS YEAR,
    (MI.YEAR_MAX - ED.SIZE_OF_COLONY) AS YEAR_DEV,
    ED.ID
FROM
    ECOLI_DATA ED
JOIN
    MAX_INFO MI
ON
    YEAR(ED.DIFFERENTIATION_DATE) = MI.YEAR
ORDER BY
    YEAR ASC,
    YEAR_DEV ASC
```

GROUP BY

저자 별 카테고리 별 매출액 집계하기

-- [조건 1] 2022년 1월의 도서 판매 데이터를 기준으로
-- [조건 2] 저자 별, 카테고리 별 매출액(TOTAL_SALES = 판매량 * 판매가) 을 구하여,
-- [조건 3] 저자 ID(AUTHOR_ID), 저자명(AUTHOR_NAME), 카테고리(CATEGORY), 매출액(SALES) 리스트를 출력
-- [조건 4] 결과는 저자 ID를 오름차순으로, 저자 ID가 같다면 카테고리를 내림차순 정렬

```
WITH BOOK_SALES_AGG AS(
    SELECT
        BOOK_ID,
        SUM(SALES) AS 'SALES_QUANTITY'
    FROM
        BOOK_SALES
    WHERE
        SALES_DATE LIKE '2022-01%'
    GROUP BY
        BOOK_ID
)

SELECT
    B.AUTHOR_ID,
    A.AUTHOR_NAME,
    B.CATEGORY,
    SUM(B.PRICE * AGG.SALES_QUANTITY) AS 'TOTAL_SALES'
FROM
    BOOK B
JOIN
    AUTHOR A
ON
    B.AUTHOR_ID = A.AUTHOR_ID
JOIN
    BOOK_SALES_AGG AGG
ON
    B.BOOK_ID = AGG.BOOK_ID
GROUP BY
    A.AUTHOR_NAME, B.CATEGORY
ORDER BY
    B.AUTHOR_ID ASC, B.CATEGORY DESC
```

식품분류별 가장 비싼 식품의 정보 조회하기

-- [조건 1] 식품분류별로 가격이 제일 비싼 식품의 분류, 가격, 이름을 조회
-- [조건 2] 식품분류가 '과자', '국', '김치', '식용유'인 경우만 출력
-- [조건 3] 결과는 식품 가격을 기준으로 내림차순 정렬

-- 방법 1

```
SELECT CATEGORY,
    MAX(PRICE) AS MAX_PRICE,
    (SELECT PRODUCT_NAME
     FROM FOOD_PRODUCT FP2
     WHERE FP2.CATEGORY = FP1.CATEGORY
        AND FP2.PRICE = MAX(FP1.PRICE)) AS PRODUCT_NAME
FROM
    FOOD_PRODUCT FP1
```

```

WHERE
    CATEGORY IN ('과자', '국', '김치', '식용유')
GROUP BY
    CATEGORY
ORDER BY
    MAX_PRICE DESC

-- 방법 2

SELECT
    FP1.CATEGORY,
    FP1.PRICE AS MAX_PRICE,
    FP1.PRODUCT_NAME
FROM FOOD_PRODUCT FP1
WHERE FP1.CATEGORY IN ('과자', '국', '김치', '식용유')
    AND FP1.PRICE = (
        SELECT MAX(FP2.PRICE)
        FROM FOOD_PRODUCT FP2
        WHERE FP2.CATEGORY = FP1.CATEGORY
    )
ORDER BY FP1.PRICE DESC;

```

GROUP BY의 역할

GROUP BY 는 데이터를 그룹별로 묶어주는 역할을 합니다. 예를 들어, CATEGORY 를 기준으로 GROUP BY 를 하면 CATEGORY 값이 같은 데이터끼리 묶이게 됩니다. 이렇게 묶인 데이터들에 대해 집계 함수를 적용해서 계산할 수 있습니다. 예를 들면:

- MAX(PRICE) : 각 그룹에서 가장 비싼 가격
- COUNT(*) : 각 그룹에 속한 데이터의 개수

하지만 중요한 점은 GROUP BY로 묶인 데이터에서는 그 그룹의 대표 값을 하나만 반환해야 한다는 것입니다.

문제: 왜 PRODUCT_NAME 이 에러를 발생시키는지

작성하신 코드에서는 CATEGORY 를 기준으로 데이터를 그룹화(GROUP BY)하셨습니다. 이렇게 하면 SQL은 CATEGORY 별로 데이터를 묶게 됩니다. 그런데 PRODUCT_NAME 은 묶인 데이터 안에서 여러 값이 있을 수 있습니다.

예를 들어, "과자"라는 그룹에 여러 제품이 있다면, SQL 입장에서 어느 PRODUCT_NAME 을 반환해야 할지 모호합니다. 이때 SQL은 이렇게 판단합니다:

"GROUP BY로 묶었는데, 묶인 데이터 중 어떤 제품 이름(PRODUCT_NAME)을 보여줄지 명확하지 않다. 그래서 에러를 낼 수밖에 없다."

이런 상황이 바로 GROUP BY 에러를 발생시키는 원인입니다.

GROUP BY에서 반드시 지켜야 할 규칙

SELECT 에서 사용하는 모든 컬럼은 아래 중 하나여야 합니다:

27. GROUP BY에 포함된 컬럼 (CATEGORY)

28. 집계 함수로 처리된 값 (MAX(PRICE))

각 방법별 해설

- 방법 1
 - 데이터 필터링:
 - WHERE CATEGORY IN ('과자', '국', '김치', '식용유')에 의해 해당 카테고리만 추출됩니다.
 - 그룹화:
 - GROUP BY CATEGORY에 의해 각 카테고리별로 데이터를 묶습니다.
 - 최대 가격 계산:
 - 각 그룹에서 MAX(PRICE)를 계산하여 MAX_PRICE 컬럼을 생성합니다.
 - 서브쿼리를 통해 제품 이름 가져오기:
 - 서브쿼리는 각 카테고리에서 최대 가격(MAX_PRICE)에 해당하는 제품의 이름을 추출합니다.
 - 정렬:
 - ORDER BY MAX_PRICE DESC에 의해 가장 비싼 제품이 속한 카테고리가 먼저 출력됩니다.
- 방법 2
 - 각 카테고리에서 최대 가격(MAX(PRICE))을 먼저 구하고, 이를 만족하는 PRODUCT_NAME을 가져옴
 - 각 그룹에서 가장 비싼 가격을 가진 제품 이름을 가져오기 위해 서브쿼리를 사용

대여 횟수가 많은 자동차들의 월별 대여 횟수 구하기

```
-- [조건 1] 대여 시작일을 기준으로 2022년 8월부터 2022년 10월까지 총 대여 횟수가 5회 이상인 자동차들
-- [조건 2] 해당 기간 동안의 월별 자동차 ID 별 총 대여 횟수(컬럼명: RECORDS) 리스트를 출력
-- [조건 3] 결과는 월을 기준으로 오름차순 정렬하고, 월이 같다면 자동차 ID를 기준으로 내림차순 정렬
-- [조건 4] 특정 월의 총 대여 횟수가 0인 경우에는 결과에서 제외
```

```
WITH RENT_COUNTS AS (
    SELECT
        CAR_ID
    FROM
        CAR_RENTAL_COMPANY_RENTAL_HISTORY
    WHERE
        START_DATE BETWEEN '2022-08-01' AND '2022-10-31'
    GROUP BY
        CAR_ID
    HAVING
        COUNT(*) >= 5
)

SELECT
    MONTH(START_DATE) AS 'MONTH',
    CAR_ID,
    COUNT(*) AS 'RECORDS'
FROM
    CAR_RENTAL_COMPANY_RENTAL_HISTORY
WHERE
    (CAR_ID IN (SELECT * FROM RENT_COUNTS))
    & (START_DATE BETWEEN '2022-08-01' AND '2022-10-31')
GROUP BY
    MONTH(START_DATE), CAR_ID
HAVING
    COUNT(*) > 0
ORDER BY
```

MONTH ASC, CAR_ID DESC

자동차 대여 기록에서 대여중 / 대여 가능 여부 구분하기

- [조건 1] 2022년 10월 16일에 대여 중인 자동차인 경우 '대여중' 이라고 표시하고, 대여 중이지 않은 자동차인 경우
- [조건 2] 자동차 ID와 AVAILABILITY 리스트를 출력하는 SQL문을 작성
- [조건 3] 반납 날짜가 2022년 10월 16일인 경우에도 '대여중'으로 표시
- [조건 4] 결과는 자동차 ID를 기준으로 내림차순 정렬

```
# SELECT
#     CAR_ID,
#     (CASE
#         WHEN MAX(END_DATE) < '2022-10-16' THEN '대여 가능'
#         WHEN (MAX(START_DATE) < '2022-10-16'
#             & MAX(END_DATE) >= '2022-10-16') THEN '대여중'
#         END) AS 'AVAILABILITY'
# FROM
#     CAR_RENTAL_COMPANY_RENTAL_HISTORY
# GROUP BY
#     CAR_ID
# ORDER BY
#     CAR_ID DESC;

WITH RENTED AS(
    SELECT
        CAR_ID,
        MAX(CASE
            WHEN '2022-10-16' BETWEEN START_DATE AND END_DATE THEN 1
            # WHEN START_DATE > '2022-10-16' AND END_DATE < '2022-10-16'
            ELSE 0
        END) AS 'RENTED'
    FROM
        CAR_RENTAL_COMPANY_RENTAL_HISTORY
    GROUP BY
        CAR_ID
)

SELECT DISTINCT
    CAR_ID,
    (CASE
        WHEN CAR_ID IN (SELECT
                            CAR_ID
                        FROM
                            RENTED
                        WHERE
                            RENTED = 1) THEN '대여중'
        ELSE '대여 가능' END) AS 'AVAILABILITY'
FROM
    CAR_RENTAL_COMPANY_RENTAL_HISTORY
ORDER BY
```

CAR_ID DESC

자동차 종류 별 특정 옵션이 포함된 자동차 수 구하기

-- [조건 1] '통풍시트', '열선시트', '가죽시트' 중 하나 이상의 옵션이 포함된 자동차가 자동차 종류 별로 몇 대인지 ;
-- [조건 2] 자동차 수에 대한 컬럼명은 CARS로 지정
-- [조건 3] 결과는 자동차 종류를 기준으로 오름차순 정렬

```
WITH OPTION_INCLUDED AS(  
    SELECT  
        CAR_TYPE,  
        CASE  
            WHEN ((OPTIONS LIKE '%통풍시트%')  
                  OR (OPTIONS LIKE '%열선시트%')  
                  OR (OPTIONS LIKE '%가죽시트%')) THEN 1  
            ELSE 0  
        END AS OPTION_YES  
    FROM  
        CAR_RENTAL_COMPANY_CAR  
)
```

```
SELECT  
    CAR_TYPE,  
    SUM(OPTION_YES) AS CARS  
FROM  
    OPTION_INCLUDED  
GROUP BY  
    CAR_TYPE  
ORDER BY  
    CAR_TYPE
```

성분으로 구분한 아이스크림 총 주문량

-- [조건 1] 상반기 동안 각 아이스크림 성분 타입과 성분 타입에 대한 아이스크림의 총주문량을 총주문량이 작은 순서대로
-- [조건 2] 총주문량을 나타내는 컬럼명은 TOTAL_ORDER로 지정

```
SELECT  
    II.INGREDIENT_TYPE,  
    SUM(TOTAL_ORDER) AS TOTAL_ORDER  
FROM  
    FIRST_HALF FH  
JOIN  
    ICECREAM_INFO II  
ON  
    FH.FLAVOR = II.FLAVOR  
GROUP BY  
    II.INGREDIENT_TYPE
```

```
ORDER BY
TOTAL_ORDER ASC
```

카테고리 별 도서 판매량 집계하기

```
-- [조건 1] 2022년 1월의 카테고리 별 도서 판매량을 합산
-- [조건 2] 카테고리(CATEGORY), 총 판매량(TOTAL_SALES) 리스트를 출력
-- [조건 3] 결과는 카테고리명을 기준으로 오름차순 정렬
```

```
SELECT
    B.CATEGORY,
    SUM(BS.SALES) AS 'TOTAL_SALES'
FROM
    BOOK_SALES BS
JOIN
    BOOK B
ON
    BS.BOOK_ID = B.BOOK_ID
WHERE
    SALES_DATE LIKE '2022-01%'
GROUP BY
    CATEGORY
ORDER BY
    CATEGORY ASC
```

즐거찾기가 가장 많은 식당 정보 출력하기

```
-- [조건 1] 음식종류별로 즐겨찾기수가 가장 많은 식당의 음식 종류, ID, 식당 이름, 즐겨찾기수를 조회하는 SQL문을 작
-- [조건 2] 결과는 음식 종류를 기준으로 내림차순 정렬
```

```
WITH MAX_FAVOR_BY_TYPE AS(
    SELECT
        FOOD_TYPE,
        MAX(FAVORITES) AS FAVORITES
    FROM
        REST_INFO
    GROUP BY
        FOOD_TYPE
)
```

```
SELECT
    MF.FOOD_TYPE,
    RI.REST_ID,
    RI.REST_NAME,
    MF.FAVORITES
FROM
    MAX_FAVOR_BY_TYPE MF
JOIN
    REST_INFO RI
ON
    (MF.FOOD_TYPE = RI.FOOD_TYPE)
    & (MF.FAVORITES = RI.FAVORITES)
```

```
ORDER BY
    FOOD_TYPE DESC
```

조건에 맞는 사용자와 총 거래금액 조회하기

```
-- [조건 1] 완료된 중고 거래
-- [조건 2] 거래 총금액이 70만 원 이상인 사람의 회원 ID, 닉네임, 총거래금액을 조회
-- [조건 3] 결과는 총거래금액을 기준으로 오름차순 정렬
```

```
SELECT
    U.USER_ID,
    U.NICKNAME,
    SUM(B.PRICE) AS 'TOTAL_SALES'
FROM
    USED_GOODS_BOARD B
JOIN
    USED_GOODS_USER U
ON
    B.WRITER_ID = U.USER_ID
WHERE
    B.STATUS = 'DONE'
GROUP BY
    USER_ID, NICKNAME
HAVING
    SUM(B.PRICE) >= 700000
ORDER BY
    TOTAL_SALES ASC
```

진료과별 총 예약 횟수 출력하기

```
-- [조건 1] 2022년 5월에 예약한 환자 수를 진료과코드 별로 조회
-- [조건 2] 컬럼명은 '진료과 코드', '5월예약건수'로 지정
-- [조건 3] 결과는 진료과별 예약한 환자 수를 기준으로 오름차순 정렬, 진료과 코드를 기준으로 오름차순 정렬
```

```
SELECT
    MCDP_CD AS '진료과코드',
    COUNT(*) AS '5월예약건수'
FROM
    APPOINTMENT
WHERE
    APNT_YMD LIKE '2022-05%'
GROUP BY
    MCDP_CD
ORDER BY
    5월예약건수 ASC,
    MCDP_CD ASC
```

고양이와 개는 몇 마리 있을까

-- [조건 1] 고양이와 개가 각각 몇 마리인지 조회하는 SQL문을 작성
-- [조건 2] 고양이를 개보다 먼저 조회

```
SELECT
    ANIMAL_TYPE,
    COUNT(*) AS 'count'
FROM
    ANIMAL_INS
GROUP BY
    ANIMAL_TYPE
ORDER BY
    ANIMAL_TYPE ASC
```

동명 동물 수 찾기

-- [조건 1] 이름 중 두 번 이상 쓰인 이름과 해당 이름이 쓰인 횟수를 조회
-- [조건 2] 결과는 이름이 없는 동물은 집계에서 제외
-- [조건 3] 결과는 이름 순으로 조회

```
SELECT
    NAME,
    COUNT(*) AS 'COUNT'
FROM
    ANIMAL_INS
WHERE
    NAME IS NOT NULL
GROUP BY
    NAME
HAVING
    COUNT(*) >= 2
ORDER BY
    NAME
```

년, 월, 성별 별 상품 구매 회원 수 구하기

-- [조건 1] 년, 월, 성별 별로 상품을 구매한 회원수를 집계
-- [조건 2] 결과는 년, 월, 성별을 기준으로 오름차순 정렬
-- [조건 3] 성별 정보가 없는 경우 결과에서 제외

```
SELECT
    YEAR(S.SALES_DATE) AS YEAR,
    MONTH(S.SALES_DATE) AS MONTH,
    U.GENDER,
    COUNT(DISTINCT S.USER_ID) AS USERS
FROM
    ONLINE_SALE S
JOIN
    USER_INFO U
ON
    S.USER_ID = U.USER_ID
```

```

WHERE
    GENDER IS NOT NULL
GROUP BY
    YEAR(S.SALES_DATE), MONTH(S.SALES_DATE), U.GENDER
ORDER BY
    YEAR, MONTH, GENDER

```

- COUNT 에다가 DISTINCT 를 쓰고 싶을 때는, COUNT 함수 안에다가 DISTINCT 컬럼명을 넣으면 된다.

입양 시각 구하기(1)

```

-- [조건 1] 09:00부터 19:59까지, 각 시간대별로 입양이 몇 건이나 발생했는지 조회
-- [조건 2] 결과는 시간대 순으로 정렬

```

```

SELECT
    HOUR(DATETIME) AS 'HOUR',
    COUNT(*) AS 'COUNT'
FROM
    ANIMAL_OUTS
WHERE
    HOUR(DATETIME) BETWEEN 9 AND 19
GROUP BY
    HOUR(DATETIME)
ORDER BY
    HOUR

```

입양 시각 구하기(2)

```

-- [조건 1] 0시부터 23시까지, 각 시간대별로 입양이 몇 건이나 발생했는지 조회
-- [조건 2] 결과는 시간대 순으로 정렬

```

```

-- 0~23까지의 숫자 시퀀스 생성
WITH RECURSIVE Hours AS (
    SELECT 0 AS HOUR
    UNION ALL
    SELECT HOUR + 1
    FROM Hours
    WHERE HOUR < 23
),

DATA_COUNTS AS(
    SELECT
        HOUR(DATETIME) AS 'HOUR',
        COUNT(*) AS 'COUNT'
    FROM
        ANIMAL_OUTS
    GROUP BY
        HOUR(DATETIME)
    ORDER BY
        HOUR
)

```

```
SELECT
    H.HOUR,
    IF(DC.COUNT IS NULL, 0, DC.COUNT) AS 'COUNT'
FROM
    Hours H
LEFT JOIN
    DATA_COUNTS DC
ON
    H.HOUR = DC.HOUR
ORDER BY
    HOUR
```

- Recursive CTE 사용 방법에 대해서 기억해두자
- 이를 통해 0~23의 Sequence 값을 갖는 테이블을 만들고, JOIN을 수행한다

가격대 별 상품 개수 구하기

-- [조건 1] 만원 단위의 가격대 별로 상품 개수를 출력
 -- [조건 2] 컬럼명은 각각 컬럼명은 PRICE_GROUP, PRODUCTS로 지정
 -- [조건 3] 가격대 정보는 각 구간의 최소금액(10,000원 이상 ~ 20,000 미만인 구간인 경우 10,000)으로 표시
 -- [조건 4] 결과는 가격대를 기준으로 오름차순 정렬

```
SELECT
    FLOOR(PRICE / 10000) * 10000 AS PRICE_GROUP,
    COUNT(*) AS PRODUCTS
FROM
    PRODUCT
GROUP BY
    PRICE_GROUP
ORDER BY
    PRICE_GROUP ASC
```

- 반올림, 올림, 내림, 자르기 등의 함수를 익혀두자.

함수	설명	결과 예시 (123.456)
ROUND	지정한 소수점 자리에서 반올림	123.46
CEIL	소수점 올림, 더 큰 정수 반환	124
FLOOR	소수점 내림, 더 작은 정수 반환	123
TRUNCATE	지정한 소수점 자리 이후 값 자르기	123.45

언어별 개발자 분류하기

-- [조건 1] DEVELOPERS 테이블에서 GRADE별 개발자의 정보를 조회
 -- [조건 2] GRADE는 다음과 같이 정해집니다.
 -- A : Front End 스킬과 Python 스킬을 함께 가지고 있는 개발자
 -- B : C# 스킬을 가진 개발자
 -- C : 그 외의 Front End 개발자
 -- [조건 3] GRADE가 존재하는 개발자의 GRADE, ID, EMAIL을 조회
 -- [조건 4] 결과는 GRADE와 ID를 기준으로 오름차순 정렬해 주세요.

```

WITH FRONT_END AS(
    SELECT
        SUM(CODE)
    FROM
        SKILLCODES
    WHERE
        CATEGORY = 'FRONT END'
),

PYTHON AS(
    SELECT
        CODE
    FROM
        SKILLCODES
    WHERE
        NAME = 'Python'
),

C_SHARP AS(
    SELECT
        CODE
    FROM
        SKILLCODES
    WHERE
        NAME = 'C#'
),

DEVELOPERS_WITH_GRADE AS (
    SELECT
        CASE
            WHEN (SKILL_CODE & (SELECT * FROM FRONT_END) > 0)
                & (SKILL_CODE & (SELECT * FROM PYTHON) > 0) THEN 'A'
            WHEN (SKILL_CODE & (SELECT * FROM C_SHARP) > 0) THEN 'B'
            WHEN (SKILL_CODE & (SELECT * FROM FRONT_END) > 0)
                & ~(SKILL_CODE & (SELECT * FROM PYTHON) > 0) THEN 'C'
        END AS GRADE,
        ID,
        EMAIL
    FROM
        DEVELOPERS
)

SELECT
    GRADE,
    ID,
    EMAIL
FROM
    DEVELOPERS_WITH_GRADE
WHERE
    GRADE IS NOT NULL
ORDER BY
    GRADE ASC, ID ASC

```

- 여러가지 스킬셋에 대해 SUM을 하거나 BIT_OR을 곱해서 스킬셋을 비트로 표현 가능

- 비트 연산자(&)를 사용해서 특정 스킬 보유 여부 확인 가능
- 이를 정리한 후, GRADE를 확인하기 위한 연산 수행
- SELECT 절에서 CASE 등으로 연산 수행할 경우, WHERE절을 통해 조건을 걸 수 없으므로 CTE를 이용해서 한번 더 정리하자

조건에 맞는 사원 정보 조회하기

- [조건 1] 2022년도 한해 평가 점수가 가장 높은 사원 정보를 조회
- [조건 2] 평가 점수가 가장 높은 사원들의 점수, 사번, 성명, 직책, 이메일을 조회
- [조건 3] 2022년도의 평가 점수는 상,하반기 점수의 합을 의미
- [조건 4] 평가 점수를 나타내는 컬럼의 이름은 SCORE로

```
WITH BEST_EMPLOYEE AS (
    SELECT
        SUM(SCORE) AS SCORE,
        EMP_NO
    FROM
        HR_GRADE
    WHERE
        YEAR = '2022'
    GROUP BY
        EMP_NO
    ORDER BY
        SUM(SCORE) DESC
    LIMIT 1
)
```

```
SELECT
    BE.SCORE,
    BE.EMP_NO,
    E.EMP_NAME,
    E.POSITION,
    E.EMAIL
FROM
    BEST_EMPLOYEE BE
JOIN
    HR_EMPLOYEES E
ON
    BE.EMP_NO = E.EMP_NO
```

연간 평가점수에 해당하는 평가 등급 및 성과금 조회하기

- [조건 1] 사원별 성과금 정보를 조회
- [조건 2] 평가 점수별 등급과 등급에 따른 성과금 정보가 아래와 같음
- [조건 3] 사번, 성명, 평가 등급, 성과금을 조회하는 SQL문을 작성
- [조건 4] 평가등급의 컬럼명은 GRADE로, 성과금의 컬럼명은 BONUS로
- [조건 5] 결과는 사번 기준으로 오름차순 정렬

```
WITH EMP_GRADE AS(
```

```

SELECT
    EMP_NO,
    CASE
        WHEN AVG(SCORE) >= 96 THEN 'S'
        WHEN (AVG(SCORE) >= 90) AND (AVG(SCORE) < 96) THEN 'A'
        WHEN (AVG(SCORE) >= 80) AND (AVG(SCORE) < 90) THEN 'B'
        ELSE 'C'
    END AS GRADE
FROM
    HR_GRADE
GROUP BY
    EMP_NO
)

```

```

SELECT
    EG.EMP_NO,
    E.EMP_NAME,
    EG.GRADE,
    CASE
        WHEN EG.GRADE = 'S' THEN (E.SAL*0.2)
        WHEN EG.GRADE = 'A' THEN (E.SAL*0.15)
        WHEN EG.GRADE = 'B' THEN (E.SAL*0.1)
        ELSE 0
    END AS BONUS
FROM
    EMP_GRADE EG
JOIN
    HR_EMPLOYEES E
ON
    EG.EMP_NO = E.EMP_NO
ORDER BY
    EG.EMP_NO ASC

```

- 문제에서 반기 보너스인지 연보너스인지에 대한 언급이 없지만, 연평균 점수로 계산

부서별 평균 연봉 조회하기

- [조건 1] HR_DEPARTMENT와 HR_EMPLOYEES 테이블을 이용해 부서별 평균 연봉을 조회
- [조건 2] 부서별로 부서 ID, 영문 부서명, 평균 연봉을 조회하는 SQL문을 작성
- [조건 3] 평균연봉은 소수점 첫째 자리에서 반올림
- [조건 4] 컬럼명은 AVG_SAL로
- [조건 5] 결과는 부서별 평균 연봉을 기준으로 내림차순 정렬

```

SELECT
    D.DEPT_ID,
    D.DEPT_NAME_EN,
    ROUND(AVG(SAL)) AS AVG_SAL
FROM
    HR_EMPLOYEES E
JOIN
    HR_DEPARTMENT D
ON
    E.DEPT_ID = D.DEPT_ID
GROUP BY
    D.DEPT_ID, D.DEPT_NAME_EN

```

```
ORDER BY
    ROUND(AVG(SAL)) DESC
```

노선별 평균 역 사이 거리 조회하기

```
-- [조건 1] 노선별로 노선, 총 누계 거리, 평균 역 사이 거리를 노선별로 조회
-- [조건 2] 총 누계거리는 테이블 내 존재하는 역들의 역 사이 거리의 총 합을 뜻함
-- [조건 3] 총 누계 거리와 평균 역 사이 거리의 컬럼명은 각각 TOTAL_DISTANCE, AVERAGE_DISTANCE로
-- [조건 4] 총 누계거리는 소수 둘째자리에서, 평균 역 사이 거리는 소수 셋째 자리에서 반올림 한 뒤 단위(km)를 함께
-- [조건 5] 결과는 총 누계 거리를 기준으로 내림차순 정렬
```

```
SELECT
    ROUTE,
    CONCAT(ROUND(SUM(D_BETWEEN_DIST), 1), 'km') AS TOTAL_DISTANCE,
    CONCAT(ROUND(AVG(D_BETWEEN_DIST), 2), 'km') AS AVERAGE_DISTANCE
FROM
    SUBWAY_DISTANCE
GROUP BY
    ROUTE
ORDER BY
    SUM(D_BETWEEN_DIST) DESC
```

물고기 종류 별 잡은 수 구하기

```
-- [조건 1] 물고기의 종류 별 물고기의 이름과 잡은 수를 출력
-- [조건 2] 물고기의 이름 컬럼명은 FISH_NAME, 잡은 수 컬럼명은 FISH_COUNT로
-- [조건 3] 결과는 잡은 수 기준으로 내림차순 정렬
```

```
SELECT
    COUNT(*) AS FISH_COUNT,
    FISH_NAME
FROM
    FISH_INFO FI
JOIN
    FISH_NAME_INFO FNI
ON
    FI.FISH_TYPE = FNI.FISH_TYPE
GROUP BY
    FNI.FISH_NAME
ORDER BY
    FISH_COUNT DESC
```

월별 잡은 물고기 수 구하기

```
-- [조건 1] 월별 잡은 물고기의 수와 월을 출력
-- [조건 2] 잡은 물고기 수 컬럼명은 FISH_COUNT, 월 컬럼명은 MONTH로
-- [조건 3] 결과는 월을 기준으로 오름차순 정렬
-- [조건 4] 단, 월은 숫자형태 (1~12) 로 출력하며 9 이하의 숫자는 두 자리로 출력하지 않음
```

-- [조건 5] 잡은 물고기가 없는 월은 출력하지 않음

```
WITH RECURSIVE MONTH_LIST AS (
```

```
    SELECT
        1 AS MONTH
```

```
    UNION ALL
```

```
    SELECT
        MONTH + 1
```

```
    FROM
        MONTH_LIST
```

```
    WHERE
        MONTH < 12
```

```
),
```

```
FISH_COUNT_TBL AS (
```

```
    SELECT
        MONTH(TIME) AS MONTH,
        COUNT(*) AS FISH_COUNT
```

```
    FROM
        FISH_INFO
```

```
    GROUP BY
        MONTH(TIME)
```

```
)
```

```
SELECT
```

```
    FCT.FISH_COUNT,
    ML.MONTH
```

```
FROM
    MONTH_LIST ML
```

```
LEFT JOIN
    FISH_COUNT_TBL FCT
```

```
ON
    ML.MONTH = FCT.MONTH
```

```
WHERE
    FCT.FISH_COUNT IS NOT NULL
```

특정 조건을 만족하는 물고기별 수와 최대 길이 구하기

-- [조건 1] FISH_INFO에서 평균 길이가 33cm 이상인 물고기들을 종류별로 분류하여 잡은 수, 최대 길이, 물고기의 종

-- [조건 2] 결과는 물고기 종류에 대해 오름차순으로 정렬

-- [조건 3] 10cm이하의 물고기들은 10cm로 취급하여 평균 길이를 구하기

-- [조건 4] 컬럼명은 물고기의 종류 'FISH_TYPE', 잡은 수 'FISH_COUNT', 최대 길이 'MAX_LENGTH'로

```
WITH BIG_FISH_TYPE AS (
```

```
    SELECT
        FISH_TYPE
```

```
    FROM
        FISH_INFO
```

```
    GROUP BY
        FISH_TYPE
```



```

HAVING
    AVG(IF(LENGTH IS NULL, 10, LENGTH)) > 33
)

SELECT
    COUNT(*) AS FISH_COUNT,
    MAX(IF(LENGTH IS NULL, 10, LENGTH)) AS MAX_LENGTH,
    FISH_TYPE
FROM
    FISH_INFO
WHERE
    FISH_TYPE IN (SELECT * FROM BIG_FISH_TYPE)
GROUP BY
    FISH_TYPE
ORDER BY
    FISH_TYPE ASC

```

IS NULL

경기도에 위치한 식품창고 목록 출력하기

```

-- [조건 1] 경기도에 위치
-- [조건 2] 출력 컬럼 : ID, 이름, 주소, 냉동시설 여부
-- [조건 3] 냉동시설 여부가 NULL인 경우, 'N'으로 출력
-- [조건 4] 창고 ID를 기준으로 오름차순 정렬

```

```

SELECT
    WAREHOUSE_ID,
    WAREHOUSE_NAME,
    ADDRESS,
    IF(FREEZER_YN IS NULL, 'N', FREEZER_YN)
FROM
    FOOD_WAREHOUSE
WHERE
    ADDRESS LIKE '경기%'
ORDER BY
    WAREHOUSE_ID ASC

```

이름이 없는 동물의 아이디

```

-- [조건 1] 동물의 생물 종, 이름, 성별 및 중성화 여부
-- [조건 2] 아이디 순으로 조회
-- [조건 3] NULL은 "No name"으로 표시

```

```

SELECT
    ANIMAL_TYPE,
    IF(NAME IS NULL, 'No name', NAME) AS NAME,
    SEX_UPON_INTAKE
FROM
    ANIMAL_INS

```

```
ORDER BY
  ID
```

이름이 있는 동물의 아이디

```
-- [조건 1] 이름이 있는 동물의 ID를 조회
-- [조건 2] ID는 오름차순 정렬
```

```
SELECT
  ANIMAL_ID
FROM
  ANIMAL_INS
WHERE
  NAME IS NOT NULL
ORDER BY
  ANIMAL_ID ASC
```

NULL 처리하기

```
-- [조건 1] 추출 컬럼 : 생물 종, 이름, 성별 및 중성화 여부
-- [조건 2] 아이디 순으로 조회
-- [조건 3] NULL은 "No name"으로 표시
```

```
SELECT
  ANIMAL_TYPE,
  IF(NAME IS NULL, 'No name', NAME) AS NAME,
  SEX_UPON_INTAKE
FROM
  ANIMAL_INS
ORDER BY
  ANIMAL_ID
```

나이 정보가 없는 회원 수 구하기

```
-- [조건 1] 나이 정보가 없는 회원이 몇 명인지 출력
-- [조건 2] 컬럼명은 USERS로 지정
```

```
SELECT
  COUNT(*) AS USERS
FROM
  USER_INFO
WHERE
  AGE IS NULL
```

ROOT 아이템 구하기

-- [조건 1] ROOT 아이템을 찾아 아이템 ID (ITEM_ID), 아이템 명 (ITEM_NAME)을 출력하는 SQL문을 작성
-- [조건 2] 결과는 아이템 ID를 기준으로 오름차순 정렬해 주세요.

```
SELECT
    II.ITEM_ID,
    II.ITEM_NAME
FROM
    ITEM_INFO II
JOIN
    ITEM_TREE IT
ON
    II.ITEM_ID = IT.ITEM_ID
WHERE
    PARENT_ITEM_ID IS NULL
ORDER BY
    II.ITEM_ID
```

업그레이드 할 수 없는 아이템 구하기

-- [조건 1] 더 이상 업그레이드할 수 없는 아이템의 아이템 ID (ITEM_ID), 아이템 명 (ITEM_NAME), 아이템의 희귀도
-- [조건 2] 이때 결과는 아이템 ID를 기준으로 내림차순 정렬

```
WITH NOT_FINAL_ITEM AS (
    SELECT DISTINCT
        PARENT_ITEM_ID
    FROM
        ITEM_TREE
    WHERE
        PARENT_ITEM_ID IS NOT NULL
)

SELECT
    ITEM_ID,
    ITEM_NAME,
    RARITY
FROM
    ITEM_INFO
WHERE
    ITEM_ID NOT IN (SELECT * FROM NOT_FINAL_ITEM)
ORDER BY
    ITEM_ID DESC
```

잡은 물고기의 평균 길이 구하기

```
SELECT
    ROUND(AVG(IF(LENGTH IS NULL, 10, LENGTH)), 2) AS AVERAGE_LENGTH
FROM
    FISH_INFO
```

JOIN

특정 기간동안 대여 가능한 자동차들의 대여비용 구하기

-- [조건 1] 자동차 종류가 '세단' 또는 'SUV' 인 자동차
-- [조건 2] 2022년 11월 1일부터 2022년 11월 30일까지 대여 가능
-- [조건 3] 30일간의 대여 금액이 50만원 이상 200만원 미만인 자동차
-- [조건 4] 자동차 ID, 자동차 종류, 대여 금액(컬럼명: FEE) 리스트를 출력
-- [조건 5] 결과는 대여 금액을 기준으로 내림차순 정렬하고, 자동차 종류를 기준으로 오름차순 정렬, 자동차 종류까지 같

```
WITH RENTAL_POSSIBLE AS (
    SELECT DISTINCT
        CAR_ID
    FROM
        CAR_RENTAL_COMPANY_RENTAL_HISTORY H1
    WHERE NOT EXISTS
        (
            SELECT
                1
            FROM
                CAR_RENTAL_COMPANY_RENTAL_HISTORY H2
            WHERE
                (H2.CAR_ID = H1.CAR_ID)
            AND(
                (H2.END_DATE BETWEEN '2022-11-01' AND '2022-11-30')
                OR (H2.START_DATE BETWEEN '2022-11-01' AND '2022-11-30')
                OR (H2.START_DATE <= '2022-11-01' AND H2.END_DATE >= '2022-11-30')
            )
        )
),

DISCOUNT_PROCESSED AS (

    SELECT
        CAR_TYPE,
        REPLACE(DISCOUNT_RATE, '%', '') / 100 AS DISCOUNT_RATE
    FROM
        CAR_RENTAL_COMPANY_DISCOUNT_PLAN
    WHERE
        CAR_TYPE IN ('세단', 'SUV')
        AND DURATION_TYPE = '30일 이상'
)

SELECT
    C.CAR_ID,
    C.CAR_TYPE,
```

```

DAILY_FEE * 30 * (1-DP.DISCOUNT_RATE) AS FEE
FROM
  CAR_RENTAL_COMPANY_CAR C
JOIN
  DISCOUNT_PROCESSED DP
ON
  C.CAR_TYPE = DP.CAR_TYPE
WHERE
  C.CAR_ID IN (
    SELECT * FROM RENTAL_POSSIBLE
  )
  AND C.CAR_TYPE IN ('세단', 'SUV')
  AND DAILY_FEE * 30 * (1-DP.DISCOUNT_RATE) >= 500000
  AND DAILY_FEE * 30 * (1-DP.DISCOUNT_RATE) < 2000000
ORDER BY
  FEE DESC,
  CAR_TYPE ASC,
  CAR_ID DESC

```

5월 식품들의 총매출 조회하기

-- [조건 1] 생산일자가 2022년 5월인 식품들의 식품 ID, 식품 이름, 총매출을 조회
 -- [조건 2] 결과는 총매출을 기준으로 내림차순 정렬
 -- [조건 3] 총매출이 같다면 식품 ID를 기준으로 오름차순 정렬

```

SELECT
  O.PRODUCT_ID,
  P.PRODUCT_NAME,
  SUM(O.AMOUNT * P.PRICE) AS TOTAL_SALES
FROM
  FOOD_ORDER O
JOIN
  FOOD_PRODUCT P
ON
  O.PRODUCT_ID = P.PRODUCT_ID
WHERE
  O.PRODUCE_DATE LIKE '2022-05%'
GROUP BY
  O.PRODUCT_ID,
  P.PRODUCT_NAME
ORDER BY
  TOTAL_SALES DESC,
  O.PRODUCT_ID ASC

```

주문량이 많은 아이스크림들 조회하기

-- [조건 1] 7월 아이스크림 총 주문량과 상반기의 아이스크림 총 주문량을 더한 값이 큰 순서대로 상위 3개의 맛을 조회

```

SELECT
  FLAVOR
FROM(

```

```

SELECT * FROM FIRST_HALF
UNION
SELECT * FROM JULY
) T1
GROUP BY
    FLAVOR
ORDER BY
    SUM(T1.TOTAL_ORDER) DESC
LIMIT
    3

```

조건에 맞는 도서와 저자 리스트 출력하기

```

-- [조건 1] '경제' 카테고리에 속하는 도서들
-- [조건 2] 도서 ID(BOOK_ID), 저자명(AUTHOR_NAME), 출판일(PUBLISHED_DATE) 리스트를 출력
-- [조건 3] 결과는 출판일을 기준으로 오름차순 정렬

```

```

SELECT
    B.BOOK_ID,
    A.AUTHOR_NAME,
    DATE_FORMAT(B.PUBLISHED_DATE, '%Y-%m-%d') AS PUBLISHED_DATE
FROM
    BOOK B
JOIN
    AUTHOR A
ON
    B.AUTHOR_ID = A.AUTHOR_ID
WHERE
    CATEGORY = '경제'
ORDER BY
    PUBLISHED_DATE ASC

```

그룹별 조건에 맞는 식당 목록 출력하기

```

-- [조건 1] 리뷰를 가장 많이 작성한 회원의 리뷰들을 조회
-- [조건 2] 회원 이름, 리뷰 텍스트, 리뷰 작성일이 출력되도록 작성
-- [조건 3] 결과는 리뷰 작성일을 기준으로 오름차순, 리뷰 텍스트를 기준으로 오름차순 정렬

```

```

WITH MAX_REVIEW AS (
    SELECT
        MEMBER_ID
    FROM
        REST_REVIEW
    GROUP BY
        MEMBER_ID
    ORDER BY
        COUNT(*) DESC
    LIMIT 1
)

```

```

SELECT
    M.MEMBER_NAME,
    R.REVIEW_TEXT,
    DATE_FORMAT(R.REVIEW_DATE, '%Y-%m-%d') AS REVIEW_DATE
FROM
    REST_REVIEW R
JOIN
    MEMBER_PROFILE M
ON
    R.MEMBER_ID = M.MEMBER_ID
WHERE
    R.MEMBER_ID = (SELECT * FROM MAX_REVIEW)
ORDER BY
    R.REVIEW_DATE ASC,
    R.REVIEW_TEXT ASC

```

없어진 기록 찾기

```

-- [조건 1] 천재지변으로 인해 일부 데이터가 유실되었습니다.
-- [조건 2] 입양을 간 기록은 있는데, 보호소에 들어온 기록이 없는 동물의 ID와 이름
-- [조건 3] ID 순으로 조회

```

```

SELECT
    O.ANIMAL_ID,
    O.NAME
FROM
    ANIMAL_OUTS O
WHERE NOT EXISTS(

    SELECT
        1
    FROM
        ANIMAL_INS I
    WHERE
        O.ANIMAL_ID = I.ANIMAL_ID
)
ORDER BY
    O.ANIMAL_ID

```

있었는데도 없었습니다

```

-- [조건 1] 관리자의 실수로 일부 동물의 입양일이 잘못 입력
-- [조건 2] 보호 시작일보다 입양일이 더 빠른 동물
-- [조건 3] 아이디와 이름을 조회
-- [조건 4] 보호 시작일이 빠른 순으로 조회

```

```

SELECT
    I.ANIMAL_ID,
    I.NAME

```

```

FROM
    ANIMAL_INS I
JOIN
    ANIMAL_OUTS O
ON
    I.ANIMAL_ID = O.ANIMAL_ID
WHERE
    I.DATETIME > O.DATETIME
ORDER BY
    I.DATETIME

```

오랜 기간 보호한 동물(1)

-- [조건 1] 아직 입양을 못 간 동물 중, 가장 오래 보호소에 있었던 동물 3마리
 -- [조건 2] 이름과 보호 시작일을 조회
 -- [조건 3] 결과는 보호 시작일 순으로 조회

```

SELECT
    I.NAME,
    I.DATETIME
FROM
    ANIMAL_INS I
WHERE NOT EXISTS(
    SELECT
        1
    FROM
        ANIMAL_OUTS O
    WHERE
        I.ANIMAL_ID = O.ANIMAL_ID
)
ORDER BY
    I.DATETIME ASC
LIMIT
    3

```

보호소에서 중성화한 동물

-- [조건 1] 보호소에서 중성화 수술을 거친 동물 정보를 알아보려 한다
 -- [조건 2] 보호소에 들어올 당시에는 중성화되지 않았지만, 보호소를 나갈 당시에는 중성화된 동물
 -- [조건 3] 아이디와 생물 종, 이름을 조회
 -- [조건 4] 아이디 순으로 조회
 -- [조건 5] 중성화를 거치지 않은 동물은 성별 및 중성화 여부에 Intact, 중성화를 거친 동물은 Spayed 또는 Neutered

```

SELECT
    I.ANIMAL_ID,
    I.ANIMAL_TYPE,
    I.NAME
FROM
    ANIMAL_INS I
JOIN
    ANIMAL_OUTS O
ON

```



```

I.ANIMAL_ID = O.ANIMAL_ID
WHERE
  (I.SEX_UPON_INTAKE LIKE 'INTACT%')
  AND (O.SEX_UPON_OUTCOME LIKE 'SPAYED%'
        OR O.SEX_UPON_OUTCOME LIKE 'NEUTERED%')
ORDER BY
  I.ANIMAL_ID

```

상품 별 오프라인 매출 구하기

-- [조건 1] 상품코드 별 매출액(판매가 * 판매량) 합계를 출력
 -- [조건 2] 결과는 매출액을 기준으로 내림차순, 상품코드를 기준으로 오름차순 정렬

```

SELECT
  P.PRODUCT_CODE,
  SUM((S.SALES_AMOUNT * P.PRICE)) AS SALES
FROM
  OFFLINE_SALE S
JOIN
  PRODUCT P
ON
  S.PRODUCT_ID = P.PRODUCT_ID
GROUP BY
  P.PRODUCT_CODE
ORDER BY
  SALES DESC,
  P.PRODUCT_CODE ASC

```

상품을 구매한 회원 비율 구하기

-- [조건 1] 2021년에 가입한 전체 회원들
 -- [조건 2] 상품을 구매한 회원수와 상품을 구매한 회원의 비율(=2021년에 가입한 회원 중 상품을 구매한 회원수 / 2021년에 가입한 회원수)
 -- [조건 3] 상품을 구매한 회원의 비율은 소수점 두번째자리에서 반올림
 -- [조건 4] 전체 결과는 년을 기준으로 오름차순 정렬, 월을 기준으로 오름차순 정렬

```

WITH JOINED_2021 AS (
  SELECT
    USER_ID
  FROM
    USER_INFO
  WHERE
    JOINED LIKE '2021%'
)

SELECT
  YEAR(S.SALES_DATE) AS YEAR,
  MONTH(S.SALES_DATE) AS MONTH,
  COUNT(DISTINCT S.USER_ID) AS PURCHASED_USERS,
  ROUND(COUNT(DISTINCT S.USER_ID) / (SELECT COUNT(*) FROM JOINED_2021), 1) AS PURCHASE_RATIO
FROM
  OFFLINE_SALE S

```

```

ONLINE_SALE S
JOIN
  USER_INFO U
ON
  S.USER_ID = U.USER_ID
WHERE
  S.USER_ID IN (SELECT * FROM JOINED_2021)
GROUP BY
  YEAR(SALES_DATE),
  MONTH(SALES_DATE)
ORDER BY
  YEAR,
  MONTH

```

FrontEnd 개발자 찾기

```

-- [조건 1] Front End 스킬을 가진 개발자의 정보를 조회
-- [조건 2] 조건에 맞는 개발자의 ID, 이메일, 이름, 성을 조회
-- [조건 3] 결과는 ID를 기준으로 오름차순 정렬해 주세요.

```

```

WITH FRONT_END_SKILLSET AS(

    SELECT
        SUM(CODE) AS SKILL_CODE
    FROM
        SKILLCODES
    WHERE
        CATEGORY = 'Front End'
)

SELECT
    D.ID,
    D.EMAIL,
    D.FIRST_NAME,
    D.LAST_NAME
FROM
    DEVELOPERS D
WHERE
    (D.SKILL_CODE & (SELECT SKILL_CODE FROM FRONT_END_SKILLSET)) > 0
ORDER BY
    ID

```

String, Date

자동차 평균 대여 기간 구하기

```

-- 코드를 입력하세요

```

```

-- [조건 1] 평균 대여 기간이 7일 이상인 자동차들
-- [조건 2] 자동차 ID와 평균 대여 기간(컬럼명: AVERAGE_DURATION) 리스트를 출력
-- [조건 3] 평균 대여 기간은 소수점 두번째 자리에서 반올림
-- [조건 3] 결과는 평균 대여 기간을 기준으로 내림차순 정렬, 자동차 ID를 기준으로 내림차순 정렬

```

```

SELECT
    CAR_ID,
    ROUND(AVG(DATEDIFF(END_DATE, START_DATE)),1) + 1 AS AVERAGE_DURATION
FROM
    CAR_RENTAL_COMPANY_RENTAL_HISTORY
GROUP BY
    CAR_ID
HAVING
    AVERAGE_DURATION >= 7
ORDER BY
    AVERAGE_DURATION DESC, CAR_ID DESC

```

조회수가 가장 많은 중고거래 게시판의 첨부파일 조회하기

```

-- [조건 1] USED_GOODS_BOARD와 USED_GOODS_FILE 테이블에서 조회수가 가장 높은 중고거래 게시물에 대한 첨부파일
-- [조건 2] 첨부파일 경로는 FILE ID를 기준으로 내림차순 정렬
-- [조건 3] 기본적인 파일경로는 /home/grep/src/
-- [조건 3] 게시글 ID를 기준으로 디렉토리가 구분
-- [조건 4] 파일이름은 파일 ID, 파일 이름, 파일 확장자로 구성되도록 출력
-- [조건 5] 조회수가 가장 높은 게시물은 하나만 존재

```

```

SELECT
    CONCAT('/home/grep/src/', F.BOARD_ID, '/', F.FILE_ID, F.FILE_NAME, FILE_EXT) AS FILE_PATH
FROM
    (SELECT BOARD_ID FROM USED_GOODS_BOARD ORDER BY VIEWS DESC LIMIT 1) B -- 조건 1, 3, 4,
JOIN
    USED_GOODS_FILE F
ON
    B.BOARD_ID = F.BOARD_ID
ORDER BY
    F.FILE_ID DESC -- 조건 2

```

조건에 부합하는 중고거래 상태 조회하기

```

-- [조건 1] 2022년 10월 5일에 등록된 중고거래 게시물
-- [조건 2] 게시글 ID, 작성자 ID, 게시글 제목, 가격, 거래상태를 조회
-- [조건 3] 거래상태가 SALE 이면 판매중, RESERVED이면 예약중, DONE이면 거래완료 분류하여 출력
-- [조건 4] 결과는 게시글 ID를 기준으로 내림차순 정렬

```

```

SELECT
    BOARD_ID,
    WRITER_ID,
    TITLE,
    PRICE,
    CASE STATUS WHEN 'SALE' THEN '판매중'
                WHEN 'RESERVED' THEN '예약중'
                WHEN 'DONE' THEN '거래완료'
                END AS STATUS
FROM
    USED_GOODS_BOARD
WHERE

```

```
    CREATED_DATE = '2022-10-05' -- 조건 1
ORDER BY
    BOARD_ID DESC
```

자동차 대여 기록 별 대여 금액 구하기

```
-- [조건 1] 자동차 종류가 '트럭'인 자동차의 대여 기록
-- [조건 2] 대여 기록 별로 대여 금액(컬럼명: FEE)을 구하여 대여 기록 ID와 대여 금액 리스트를 출력하는 SQL문을 작성
-- [조건 3] 결과는 대여 금액을 기준으로 내림차순 정렬, 대여 기록 ID를 기준으로 내림차순 정렬
```

```
-- Step 1 - 먼저 DISCOUNT관련 정보를 수집해서 정리해둔다.
```

WITH

```
    TRUCK_DISCOUNT AS
    (SELECT
        CAR_TYPE,
        CASE DURATION_TYPE
            WHEN '7일 이상' THEN 7
            WHEN '30일 이상' THEN 30
            WHEN '90일 이상' THEN 90
        END AS DURATION_TYPE_LOWER,

        CASE DURATION_TYPE
            WHEN '7일 이상' THEN 30
            WHEN '30일 이상' THEN 90
            WHEN '90일 이상' THEN 999999
        END AS DURATION_TYPE_UPPER,
        DISCOUNT_RATE / 100 AS DISCOUNT_RATE
    FROM
        CAR_RENTAL_COMPANY_DISCOUNT_PLAN
    WHERE
        CAR_TYPE = '트럭'
    )
```

SELECT

```
    TH.HISTORY_ID,
    ROUND(
        (TH.DAILY_FEE *
        TH.DURATION *
        (1-IFNULL(TD.DISCOUNT_RATE, 0)))) AS FEE
```

FROM

```
    (SELECT
        H.HISTORY_ID,
        H.CAR_ID,
        DATEDIFF(H.END_DATE, H.START_DATE)+1 AS DURATION,
        C.DAILY_FEE
    FROM
        CAR_RENTAL_COMPANY_RENTAL_HISTORY H
    JOIN
        CAR_RENTAL_COMPANY_CAR C
    ON
        H.CAR_ID = C.CAR_ID
    WHERE
        CAR_TYPE = '트럭') AS TH
LEFT JOIN
```

```

TRUCK_DISCOUNT TD
ON
    (TH.DURATION >= TD.DURATION_TYPE_LOWER) &
    (TH.DURATION < TD.DURATION_TYPE_UPPER)
ORDER BY
    FEE DESC,
    HISTORY_ID DESC

```

특정 옵션이 포함된 자동차 리스트 구하기

-- [조건 1] '네비게이션' 옵션이 포함된 자동차 리스트를 출력하는 SQL문을 작성
 -- [조건 2] 자동차 ID를 기준으로 내림차순 정렬

```

SELECT
    *
FROM
    CAR_RENTAL_COMPANY_CAR
WHERE
    OPTIONS LIKE '%네비게이션%'
ORDER BY
    CAR_ID DESC

```

자동차 대여 기록에서 장기/단기 대여 구분하기

-- [조건 1] 대여 시작일이 2022년 9월에 속하는 대여 기록
 -- [조건 2] 대여 기간이 30일 이상이면 '장기 대여' 그렇지 않으면 '단기 대여' 로 표시하는 컬럼(컬럼명: RENT_TYPE)
 -- [조건 3] 대여기록을 출력하는 SQL문을 작성
 -- [조건 4] 결과는 대여 기록 ID를 기준으로 내림차순 정렬

```

SELECT
    HISTORY_ID,
    CAR_ID,
    DATE_FORMAT(START_DATE, '%Y-%m-%d') AS START_DATE,
    DATE_FORMAT(END_DATE, '%Y-%m-%d') AS END_DATE,
    IF(DATEDIFF(END_DATE, START_DATE)+1 >= 30, '장기 대여', '단기 대여') AS RENT_TYPE
FROM
    CAR_RENTAL_COMPANY_RENTAL_HISTORY
WHERE
    START_DATE LIKE '2022-09%'
ORDER BY
    HISTORY_ID DESC

```

조건별로 분류하여 주문상태 출력하기

-- [조건 1] 2022년 5월 1일을 기준으로 주문 ID, 제품 ID, 출고일자, 출고여부를 조회
 -- [조건 2] 출고여부는 2022년 5월 1일까지 출고완료로 이 후 날짜는 출고 대기로 미정이면 출고미정으로 출력
 -- [조건 3] 결과는 주문 ID를 기준으로 오름차순 정렬해주세요

```

SELECT
    ORDER_ID,
    PRODUCT_ID,

```

```

DATE_FORMAT(OUT_DATE, '%Y-%m-%d'),
(CASE
  WHEN OUT_DATE <= '2022-05-01' THEN '출고완료'
  WHEN OUT_DATE > '2022-05-01' THEN '출고대기'
  WHEN OUT_DATE IS NULL THEN '출고미정' END) AS 출고여부
FROM
  FOOD_ORDER
ORDER BY
  ORDER_ID ASC

```

대여 기록이 존재하는 자동차 리스트 구하기

```

-- [조건 1] 자동차 종류가 '세단'인 자동차들 중
-- [조건 2] 10월에 대여를 시작한 기록이 있는 자동차 ID 리스트를 출력
-- [조건 2] 자동차 ID 리스트는 중복이 없어야 하며
-- [조건 3] 자동차 ID를 기준으로 내림차순 정렬

```

```

SELECT DISTINCT -- 조건 3
  H.CAR_ID
FROM
  CAR_RENTAL_COMPANY_RENTAL_HISTORY H
JOIN
  CAR_RENTAL_COMPANY_CAR C
ON
  H.CAR_ID = C.CAR_ID
WHERE
  (START_DATE LIKE '%-10-%') -- 조건 2
  & (C.CAR_TYPE = '세단') -- 조건 1
ORDER BY
  H.CAR_ID DESC

```

조건에 맞는 사용자 정보 조회하기

```

-- [조건 1] 중고 거래 게시물을 3건 이상 등록한 사용자
-- [조건 2] 사용자 ID, 닉네임, 전체주소, 전화번호를 조회
-- [조건 3] 전체 주소는 시, 도로명 주소, 상세 주소가 함께 출력
-- [조건 4] 전화번호의 경우 xxx-xxxx-xxxx 같은 형태로 하이픈 문자열(-)을 삽입하여 출력
-- [조건 5] 결과는 회원 ID를 기준으로 내림차순 정렬

```

```

SELECT
  USER_ID,
  NICKNAME,
  CONCAT(CITY, ' ', STREET_ADDRESS1, ' ', STREET_ADDRESS2) AS 전체주소,
  CONCAT(SUBSTRING(TLNO, 1,3), '-', SUBSTRING(TLNO, 4,4), '-', SUBSTRING(TLNO, 8,4)) AS
FROM
  USED_GOODS_USER
WHERE
  USER_ID IN (
    SELECT
      WRITER_ID
    FROM

```

```

        USED_GOODS_BOARD
    GROUP BY
        WRITER_ID
    HAVING
        COUNT(BOARD_ID) >= 3
    )
    ORDER BY
        USER_ID DESC

```

취소되지 않은 진료 예약 조회하기

-- [조건 1] 2022년 4월 13일 취소되지 않은 흉부외과(CS) 진료 예약 내역을 조회
 -- [조건 2] 진료예약번호, 환자이름, 환자번호, 진료과코드, 의사이름, 진료예약일시 항목이 출력되도록 작성
 -- [조건 3] 결과는 진료예약일시를 기준으로 오름차순 정렬

```

SELECT
    A.APNT_NO,
    P.PT_NAME,
    P.PT_NO,
    A.MCDP_CD,
    D.DR_NAME,
    A.APNT_YMD
FROM
    APPOINTMENT A
JOIN
    PATIENT P
ON
    A.PT_NO = P.PT_NO
JOIN
    DOCTOR D
ON
    A.MDDR_ID = D.DR_ID
WHERE
    (A.APNT_CNCL_YN = 'N')
    & (A.MCDP_CD = 'CS')
    & (A.APNT_YMD LIKE '2022-04-13%')
ORDER BY
    A.APNT_YMD ASC

```

루시와 엘라 찾기

-- [조건 1] 동물 보호소에 들어온 동물 중 이름이 Lucy, Ella, Pickle, Rogan, Sabrina, Mitty인 동물의 아C

```

SELECT
    ANIMAL_ID,
    NAME,
    SEX_UPON_INTAKE
FROM
    ANIMAL_INS
WHERE
    NAME IN ('Lucy', 'Ella', 'Pickle', 'Rogan', 'Sabrina', 'Mitty')

```

이름에 el이 들어가는 동물 찾기

-- [조건 1] 동물 보호소에 들어온 동물 이름 중, 이름에 "EL"이 들어가는 개의 아이디와 이름을 조회
-- [조건 2] 결과는 이름 순으로 조회
-- [조건 3] 단, 이름의 대소문자는 구분하지 않음

```
SELECT
    ANIMAL_ID,
    NAME
FROM
    ANIMAL_INS
WHERE
    (NAME LIKE '%EL%')
    & (ANIMAL_TYPE = 'Dog')
ORDER BY
    NAME
```

중성화 여부 파악하기

-- [조건 1] 동물의 아이디와 이름, 중성화 여부를 아이디 순으로 조회하는 SQL문을 작성
-- [조건 2] 중성화가 되어있다면 '0', 아니라면 'X'라고 표시
-- [조건 3] 중성화된 동물은 SEX_UPON_INTAKE 컬럼에 'Neutered' 또는 'Spayed'

```
SELECT
    ANIMAL_ID,
    NAME,
    CASE
        WHEN SEX_UPON_INTAKE LIKE '%Neutered%' THEN '0'
        WHEN SEX_UPON_INTAKE LIKE '%Spayed%' THEN '0'
        ELSE 'X'
    END AS 중성화
FROM
    ANIMAL_INS
```

오랜 기간 보호한 동물(2)

-- [조건 1] 입양을 간 동물 중, 보호 기간이 가장 길었던 동물 두 마리의 아이디와 이름을 조회
-- [조건 2] 결과는 보호 기간이 긴 순으로 조회

```
SELECT
    I.ANIMAL_ID,
    I.NAME
FROM
    ANIMAL_INS I
JOIN
    ANIMAL_OUTS O
ON
    I.ANIMAL_ID = O.ANIMAL_ID
WHERE
    O.DATETIME IS NOT NULL
ORDER BY
    DATEDIFF(O.DATETIME, I.DATETIME) DESC
LIMIT 2
```


카테고리 별 상품 개수 구하기

- [조건 1] PRODUCT 테이블에서 상품 카테고리 코드 (PRODUCT_CODE 앞 2자리) 별 상품 개수를 출력
- [조건 2] 결과는 상품 카테고리 코드를 기준으로 오름차순 정렬

```
SELECT
    SUBSTRING(PRODUCT_CODE, 1, 2) AS CATEGORY,
    COUNT(PRODUCT_ID) AS PRODUCTS
FROM
    PRODUCT
GROUP BY
    SUBSTRING(PRODUCT_CODE, 1, 2)
ORDER BY
    CATEGORY
```

DATETIME에서 DATE로 형 변환

- [조건 1] 각 동물의 아이디와 이름, 들어온 날짜를 조회하는 SQL문을 작성
- [조건 2] 결과는 아이디 순으로 조회

```
SELECT
    ANIMAL_ID,
    NAME,
    DATE_FORMAT(DATETIME, '%Y-%m-%d') AS 날짜
FROM
    ANIMAL_INS
ORDER BY
    ANIMAL_ID
```

연도 별 평균 미세먼지 농도 조회하기

- [조건 1] 수원 지역의 연도 별 평균 미세먼지 오염도와 평균 초미세먼지 오염도를 조회하는 SQL문을 작성
- [조건 2] 평균 미세먼지 오염도와 평균 초미세먼지 오염도의 컬럼명은 각각 PM10, PM2.5
- [조건 3] 값은 소수 셋째 자리에서 반올림
- [조건 4] 결과는 연도를 기준으로 오름차순 정렬

```
SELECT
    YEAR(YM) AS 'YEAR',
    ROUND(AVG(PM_VAL1), 2) AS 'PM10',
    ROUND(AVG(PM_VAL2), 2) AS 'PM2.5'
FROM
    AIR_POLLUTION
WHERE
    LOCATION2 = '수원'
GROUP BY
    YEAR(YM)
ORDER BY
    YEAR(YM) ASC
```

한 해에 잡은 물고기 수 구하기

-- [조건 1] 2021년도에 잡은 물고기 수를 출력
-- [조건 2] 컬럼명은 'FISH_COUNT' 로 지정

```
SELECT
    COUNT(*) AS 'FISH_COUNT'
FROM
    FISH_INFO
WHERE
    TIME LIKE '2021%'
```

분기별 분화된 대장균의 개체 수 구하기

-- [조건 1] 각 분기(QUARTER)별 분화된 대장균의 개체의 총 수(ECOLI_COUNT)를 출력
-- [조건 2] 각 분기에는 'Q' 를 붙이고 분기에 대해 오름차순으로 정렬
-- [조건 3] 대장균 개체가 분화되지 않은 분기는 없음

```
SELECT
    CONCAT(QUARTER(DIFFERENTIATION_DATE), 'Q') AS 'QUARTER',
    COUNT(ID) AS 'ECOLI_COUNT'
FROM
    ECOLI_DATA
GROUP BY
    CONCAT(QUARTER(DIFFERENTIATION_DATE), 'Q')
ORDER BY
    QUARTER
```